SAMSUNG HRM × MongoDB

HRM Data & AI
PoV Hands-on Workshop

OFFLINE WORKSHOP · SESSION 2

고객 데이터 분석 · 부분 적재 · 워크로드 검증 · PoV 설계

MongoDB. BUILD ANYTHING. CHANGE EVERYTHING.

PoV Hands-on 세션

Time	Session	Description
30분	환경 및 실습 안내	Atlas 생성, Compass 설치, 데이터셋 및 실습 절차 확인
30분	Relational Migrator	Aurora mySQL > Atlas로 data migration
60분	Lab 1: HRM 데이터 모델링	devices, device_current_state, device_events, anomaly_results 설계, Validation 및 인덱스 구성
60분	점심	
40분	Lab 2: Aggregation과 인덱스	최근 상태, 문 열림 횟수·시간, 모델·지역별 집계 및 explain() 기반 인덱스 검증
40분	Lab 3: 이벤트 품질 및 이상 감지	Kafka 이벤트 적재, Data Federation, 멱등 처리, 중복·지연·순서 역전 처리 및 3분 초과 문 열림 탐지
15분	휴식	
30분	Lab 4: Time Series Discovery	Time Series Collection 생성, 이벤트 적재·집계, TTL 적용 및 일반 컬렉션과 비교
35분	Lab 5: Vector Search/RAG Discovery	매뉴얼 임베딩 적재, Vector Search, Metadata Filtering 및 근거가 포함된 답변 생성
15분	휴식	
20분	결과 리뷰 및 PoC 반영	실습 결과 검토, 데이터 모델 확정사항, Time Series/RAG의 PoC 포함 여부 및 후속 과제 결정

세션 타임테이블

시간 (Time)	세션 (Session)	상세 내용 (Description)
10:00 - 10:30 (30분)	환경 및 실습 안내	Atlas 생성, Compass 설치, 데이터셋 및 실습 절차 확인
10:30 - 11:00 (30분)	Relational Migrator	Aurora MySQL > Atlas로 data migration
11:00 - 12:00 (60분)	Lab 1: HRM 데이터 모델링	devices, device_current_state, device_events, anomaly_results 설계, Validation 및 인덱스 구성
12:00 - 13:00 (60분)	점심 시간 (Lunch)	휴식 및 식사
13:00 - 13:40 (40분)	Lab 2: Aggregation과 인덱스	최근 상태, 문 열림 횟수·시간, 모델·지역별 집계 및 explain() 기반 인덱스 검증
13:40 - 14:20 (40분)	Lab 3: 이벤트 품질 및 이상 감지	Kafka 이벤트 적재, Data Federation, 멱등 처리, 중복·지연·순서 역전 처리 및 3분 초과 탐지
14:20 - 14:35 (15분)	휴식 (Break)	Break Time
14:35 - 15:05 (30분)	Lab 4: Time Series Discovery	Time Series Collection 생성, 이벤트 적재·집계, TTL 적용 및 일반 컬렉션과 비교
15:05 - 15:40 (35분)	Lab 5: Vector Search/RAG	매뉴얼 임베딩 적재, Vector Search, Metadata Filtering 및 근거가 포함된 답변 생성
15:40 - 15:55 (15분)	휴식 (Break)	Break Time
15:55 - 16:15 (20분)	결과 리뷰 및 PoC 반영	실습 결과 검토, 데이터 모델 확정사항, Time Series/RAG의 PoC 포함 여부 및 후속 과제 결정

일부 데이터를 적재하고, 검증할 가치를 결정합니다

1차 개념 이해 → 2차 실행 가능한 모델·워크로드 → 정식 PoV 범위 합의

모델

고객 구조를 수용

복합키 · 부분 갱신
가변 payload · 배열

워크로드

조회·갱신을 실행

최신 상태 · 최근 이력
집계 · 재처리·품질

가치

증거로 다음 단계 결정

개발 복잡도 · 응답시간
정합성 · 저장·운영 비용

종료 시점: 데이터 범위 + Golden Workload + 성공 기준 + 실행 담당자를 남깁니다

고객 자료에서 확인한 사실과 확인할 질문

운영 데이터의 값은 익명화되어 있습니다. 실제 기준은 구조·타입·키·접근 패턴·호출량입니다.

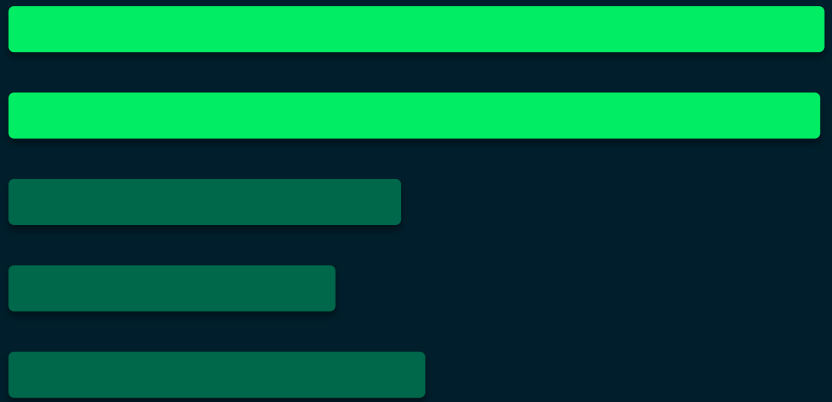
확인한 사실	PoV 설계에 주는 입력	세션에서 확인
Notification 1건	value.payload 가변 · modes 6 / supportedModes 14	제품별 필드·코드 사전
latest-value 쓰기	A–H: UPDATE 6종 / UPSERT 2종	snapshot / patch · 삭제 의미
reader 선행 조회	A·D 복합키 IN-list → 앱 병합 → 쓰기	조회 이유·배치 크기·호출률
파생 쓰기	A 이벤트 → A 갱신 + C 진단 UPSERT	원자성 필요·재처리 규칙

냉장고 JSON 1건으로 전 제품군의 필드 의미·트래픽 분포를 확정하지 않습니다

쓰기는 A·B에 집중되어 있습니다

RDS Performance Insights: 특정 1시간 writer 표본 / reader 호출률 미측정

A+B 58.9% / A–D 84.9%



합계 11,265.2 calls/s는 쓰기 SQL 호출량이며 Notification 이벤트/s가 아닙니다

냉장고 대표 범위와 쓰기 재현 범위를 함께 정합니다

아래는 시작 제안입니다. 선정된 제품군에 존재하는 Entity만 포함합니다.

현재 상태

A 우선 / B 조건부

선정 기기·리소스 복합키
원본 최신값 대조

이벤트 이력

동일 기기의 연속 구간

S3 / Kafka JSON
정상·지연·재전송 포함

확장 검증

C 파생 / D 문서 폭

대표 경로로 단계적 확대
전 제품군 수치 별도 재현

A·B를 우선 분석하되, B의 냉장고 유입과 C·D 적용 제품군은 먼저 확인합니다

부분 적재의 단위는 기기·리소스·기간입니다

무작위 행 샘플보다 상태 비교와 이벤트 순서를 검증할 수 있는 일관된 표본을 선택합니다.

구분	고객과 합의할 범위	확인 증거
스냅샷	기기 목록 × Entity × 전체 복합키 / 기준시각 T0	동일 범위 SQL·문서 건수와 값
이벤트	동일 기기·리소스의 연속 기간 [START, END)	eventId · 발생/수신 시각·순서
부하 표본	선정 제품군 비중·hot key·배치 크기·파생 비율	DB calls/s와 unique events/s 구분
분석 메타	모델·국가 / regionCode는 별도 원천 확인	코드 사전 · 필드 매핑표

추출 manifest: source / filter / 기준 시각 / 건수 / 키 목록 / 파일 hash를 기록합니다

세션마다 실행 결과와 결정을 남깁니다

10:00–16:15: 실습 · 설명 285분 + 점심 · 휴식 90분 = 총 375분 (6시간 15분)

세션	검증 질문	남길 결과
환경 + 이관 · 60분	접속하고 동일 범위 데이터를 복사했나?	연결 · 이관 log / 건수 · 타입
Lab 1 · 60분	어떤 단위로 저장 · 갱신하나?	모델 · 복합키 · Validation
Lab 2 · 3 · 각 40분	조회 · 갱신 · 재처리가 정확한가?	Pipeline · explain · 품질 결과
Lab 4 · 5 · 30/35분	시계열 / 검색이 업무에 유용한가?	비교표 · 질문셋 평가
리뷰 · 20분	정식 PoV 범위와 목표를 합의했나?	Test Card · Owner · Due

각 Lab의 완료 조건을 확인하고, 미결 항목은 마지막 PoV 카드에 연결합니다

오늘 사용할 자료와 추가 실행 입력을 확인합니다

현재 폴더의 자료를 기준으로 준비 상태를 정리했습니다.

자료	현재 확인한 상태	실행 전 준비
고객 구조·패턴	JSON 구조·Top 10·W1/W2/R1 확보	익명화 원문 JSON·코드·DDL
Atlas·이관	시작 가이드·Snapshot 가이드 확보	실제 연결·고정 표본·Job 실행
Lab 1·2	기존 교육 모델·합성 정답 장표 확보	fixture·초기화·변수 D/START/END
Kafka·품질	배치·파생 경로 구조 확인	topic 또는 replay · timer·진행 log
Time Series·RAG	아젠다·기술 실습 구성 확보	측정 JSON / 매뉴얼·질문 임베딩

합성 데이터의 기능 실습 결과와 실제 고객 데이터 PoV 결과를 구분해 기록합니다

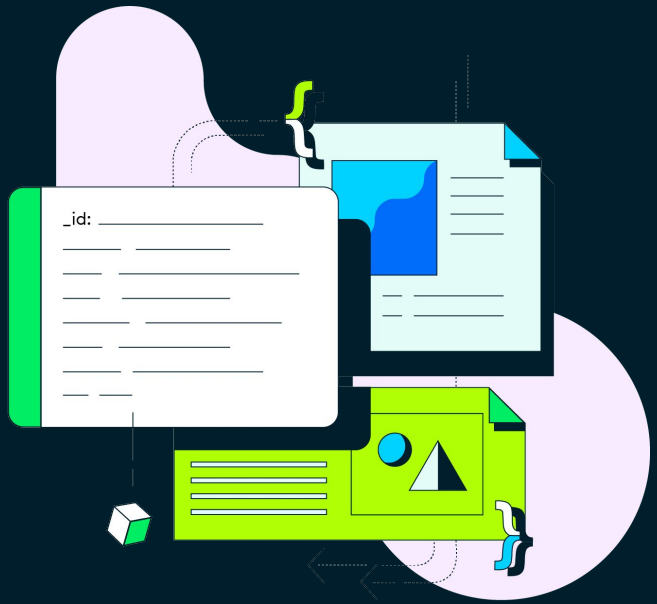
01

환경 및 실습 안내

30 MIN **Atlas 생성, Compass 설치, 데이터셋 및 실습 절차 확인**

Glossary and Access Basics

Atlas Glossary



Organization - Billable entity for Atlas deployments. Has one or more Projects.

Project - Team environment entity for Atlas, e.g. Dev, Test, Staging, Prod. Has one or more Clusters.

Cluster - Configurable cloud resources including nodes, CPU, disk, IOPS, provider, region. Has one or more Databases.

MongoDB Glossary



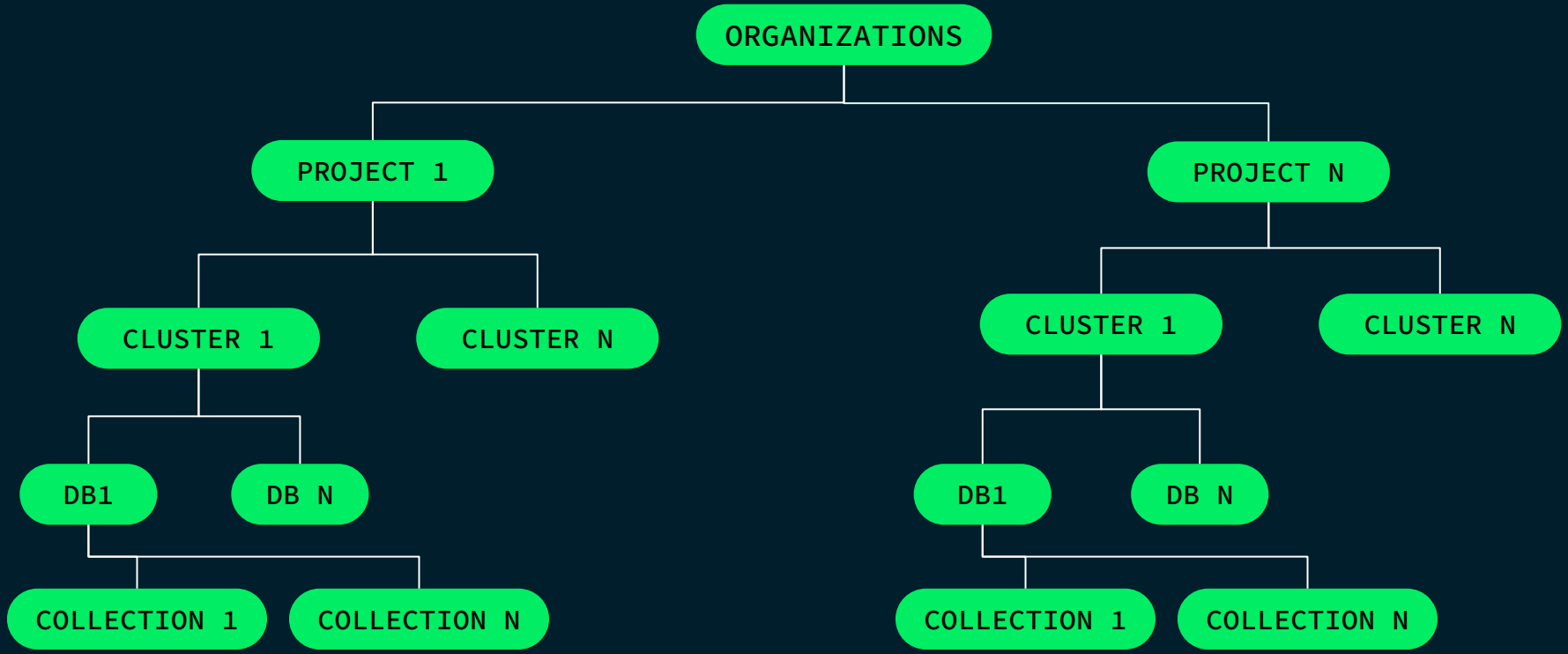
Database - Has zero or more Collections.

Collection - Has zero or more Documents.

Namespace - Synonym for “Database.Collection”

Document - Looks like JSON, stored as BSON. Has one or more sets of Fields and Values.

Atlas Hierarchy



Atlas Hierarchy on MongoDB

Organization

사용자를 관리하고 팀 및 프로젝트 간 협업을 위한 MongoDB 리소스를 담당하는 최상위 엔터티입니다. (사용자 역할, 팀 배정, 조직 수준의 접근 정책을 설정)

Project

개발, 프로덕션 같은 특정 용도 또는 환경을 위한 클러스터를 그룹화하는 조직 내 작업 공간입니다. (사용 사례 또는 환경별로 클러스터를 구성)

Cluster

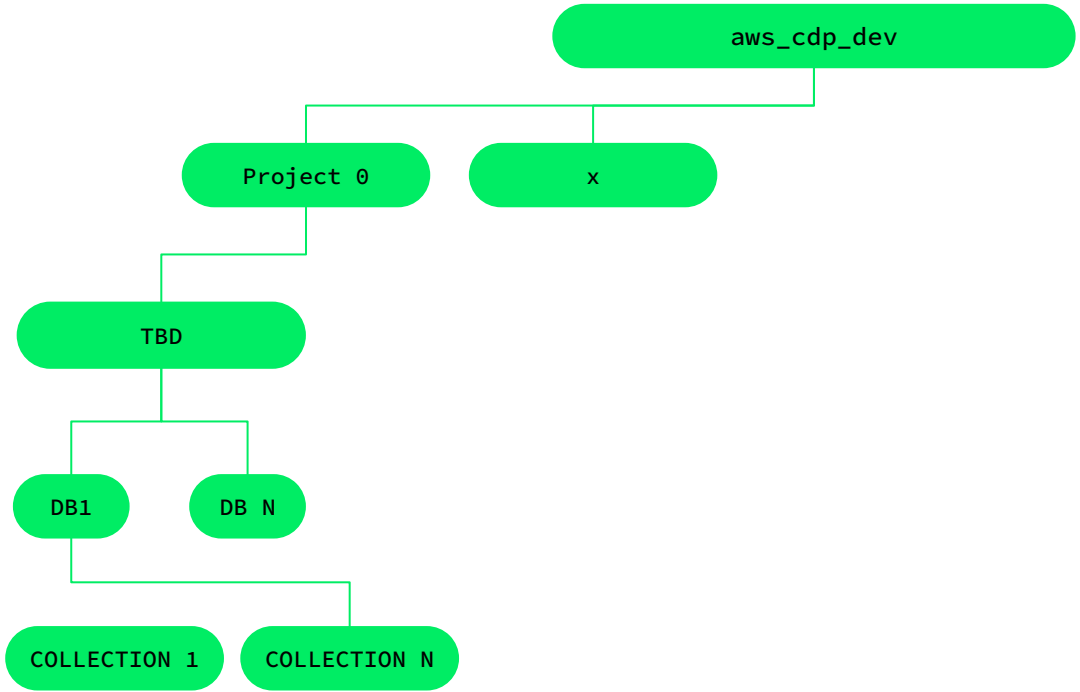
MongoDB 데이터베이스를 호스팅하고 중복성, 확장성, 성능을 제공하는 인프라입니다. (클라우드 제공자, 지역, 인스턴스 크기 및 클러스터 백업/애플리케이션 연결)

Database

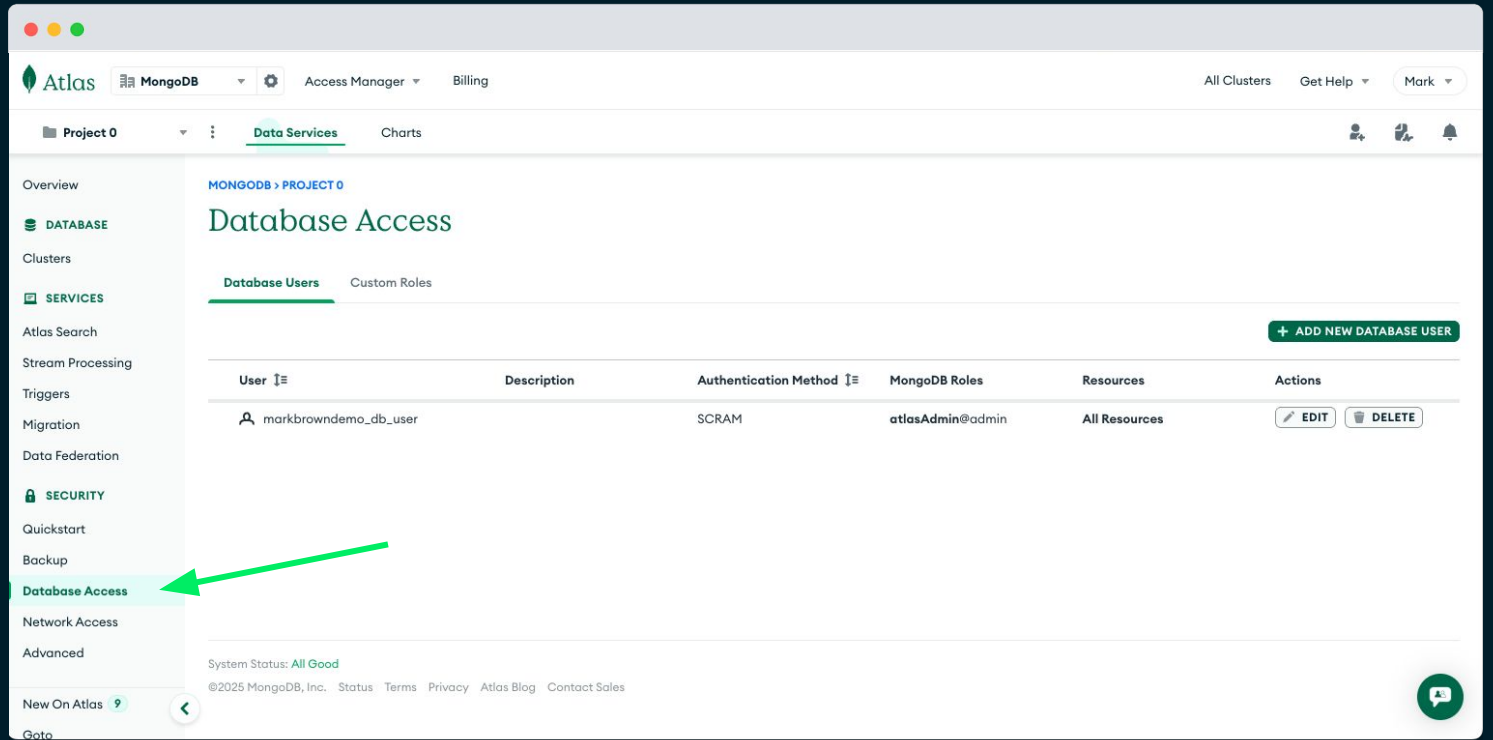
데이터를 보관할 컬렉션을 담는 컨테이너 (각 데이터베이스가 지원하는 애플리케이션이나 서비스를 식별하고 필요한 경우 데이터베이스 수준의 권한 정의)

Collection

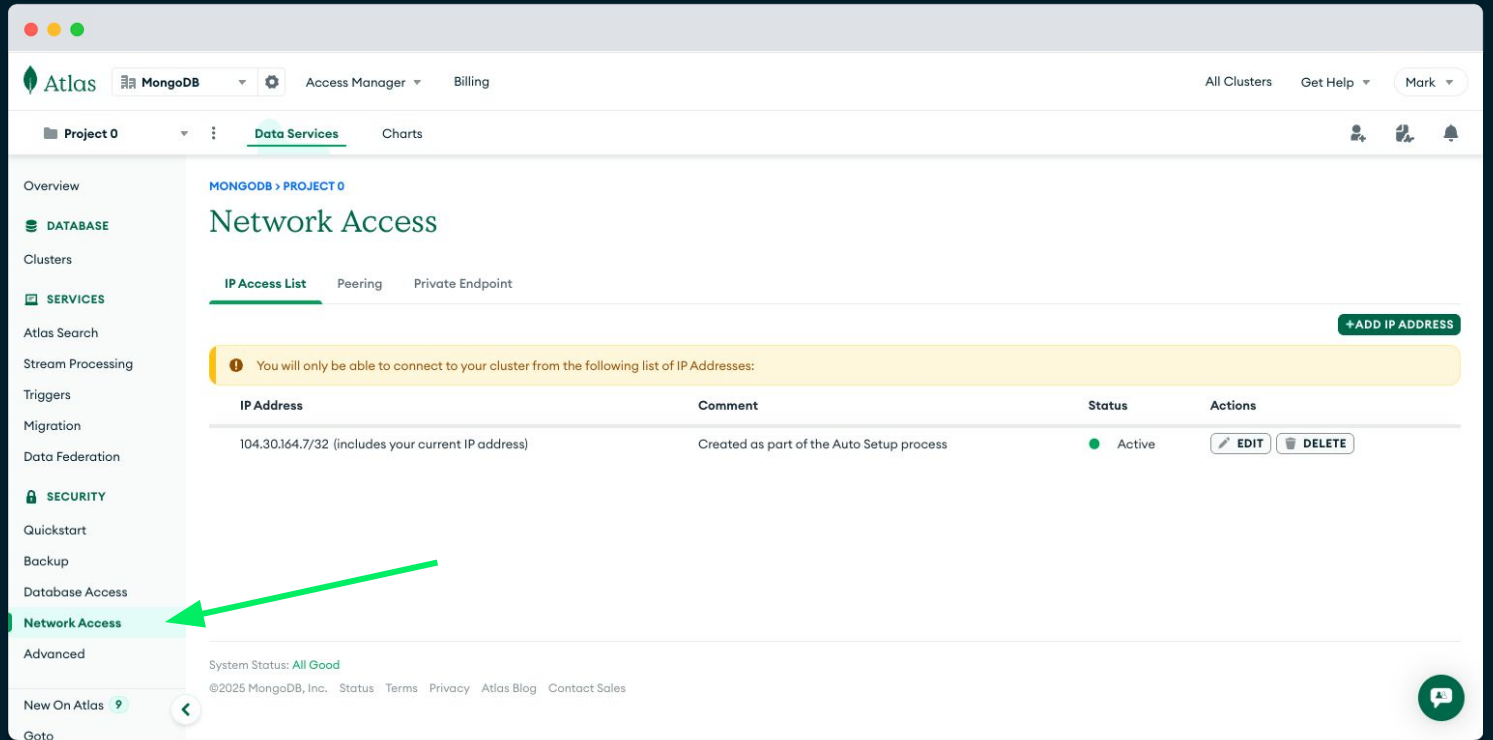
특정 유형의 데이터를 저장하는 JSON 문서들의 그룹으로, 관계형 데이터베이스의 테이블과 유사합니다.



Database Access



Network Access



Compass
설치 & 연결

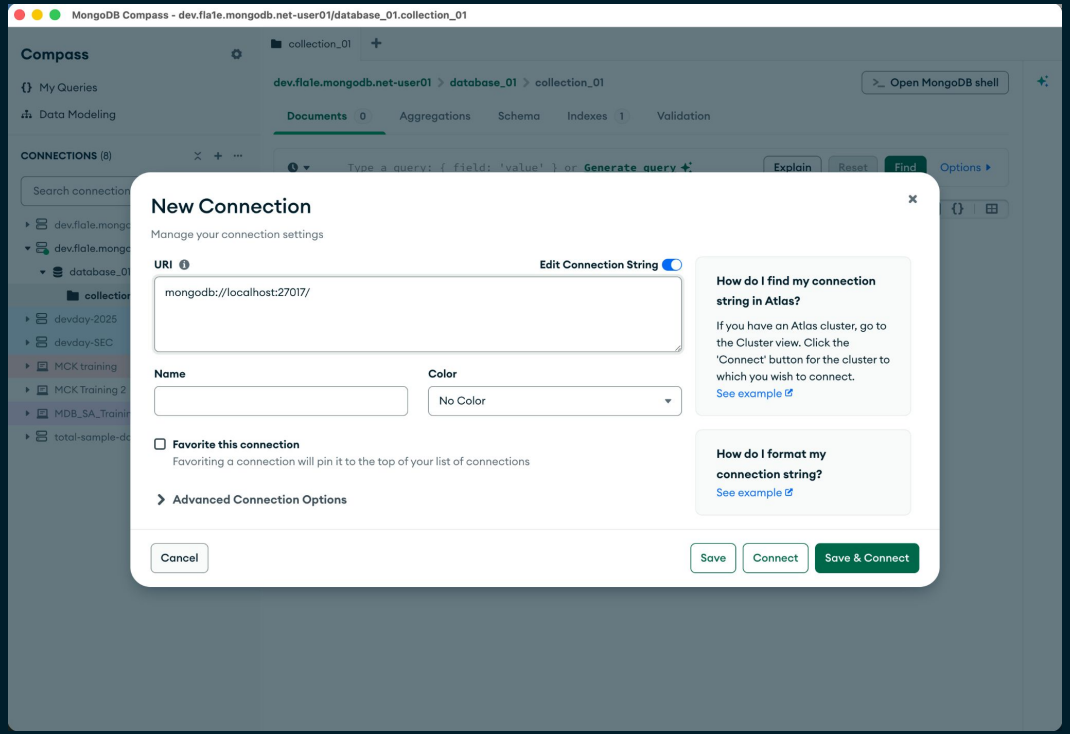
Compass 설치 & 연결

Download & Install

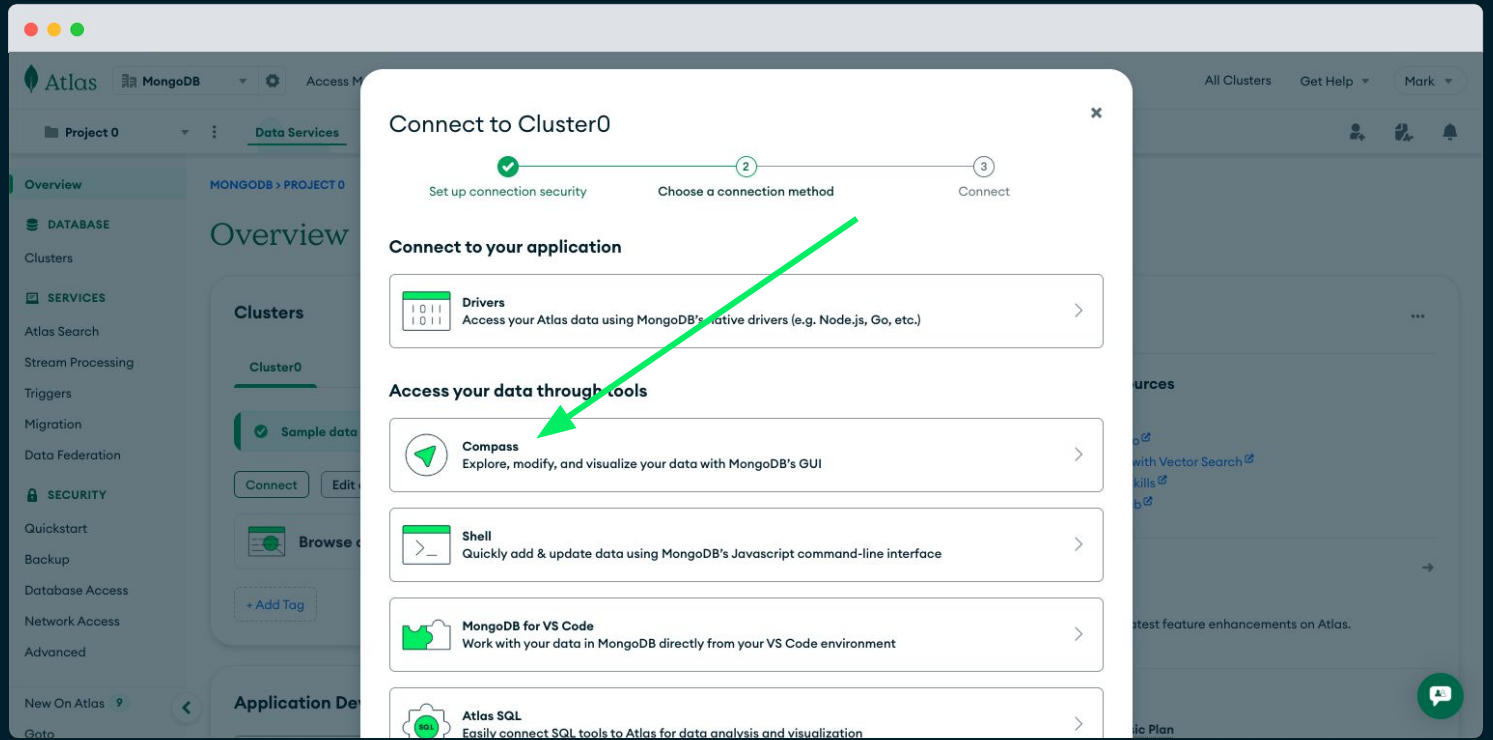
- https://www.mongodb.com/docs/compass/

Connect to Atlas

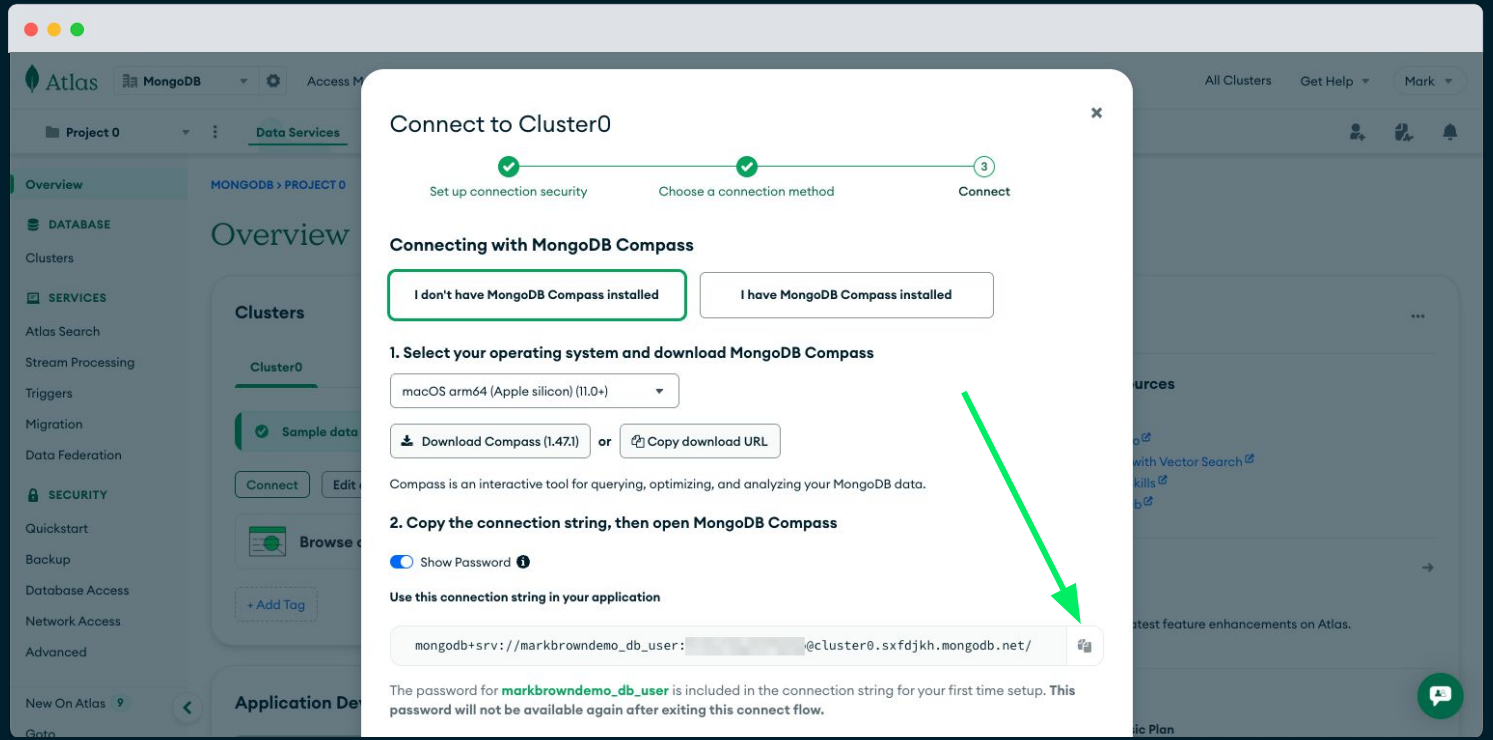
- Connections +
- New Connection URI에 Atlas connection string 입력



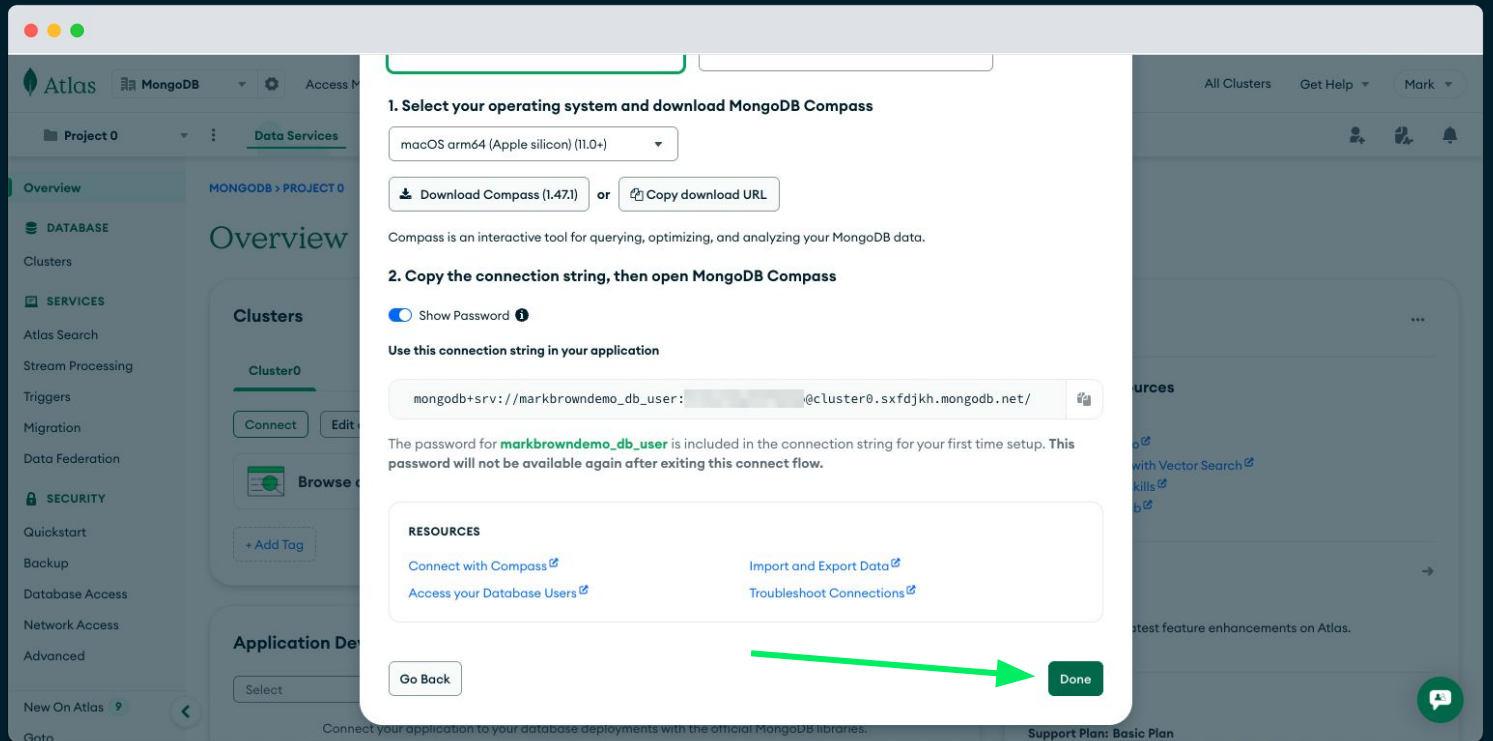
Connection Method - Compass



Copy Connection String



Scroll down to Done



02

Relational Migrator

---

30 MIN **Relational Database > Atlas로 Data Migration**

Relational Migrator

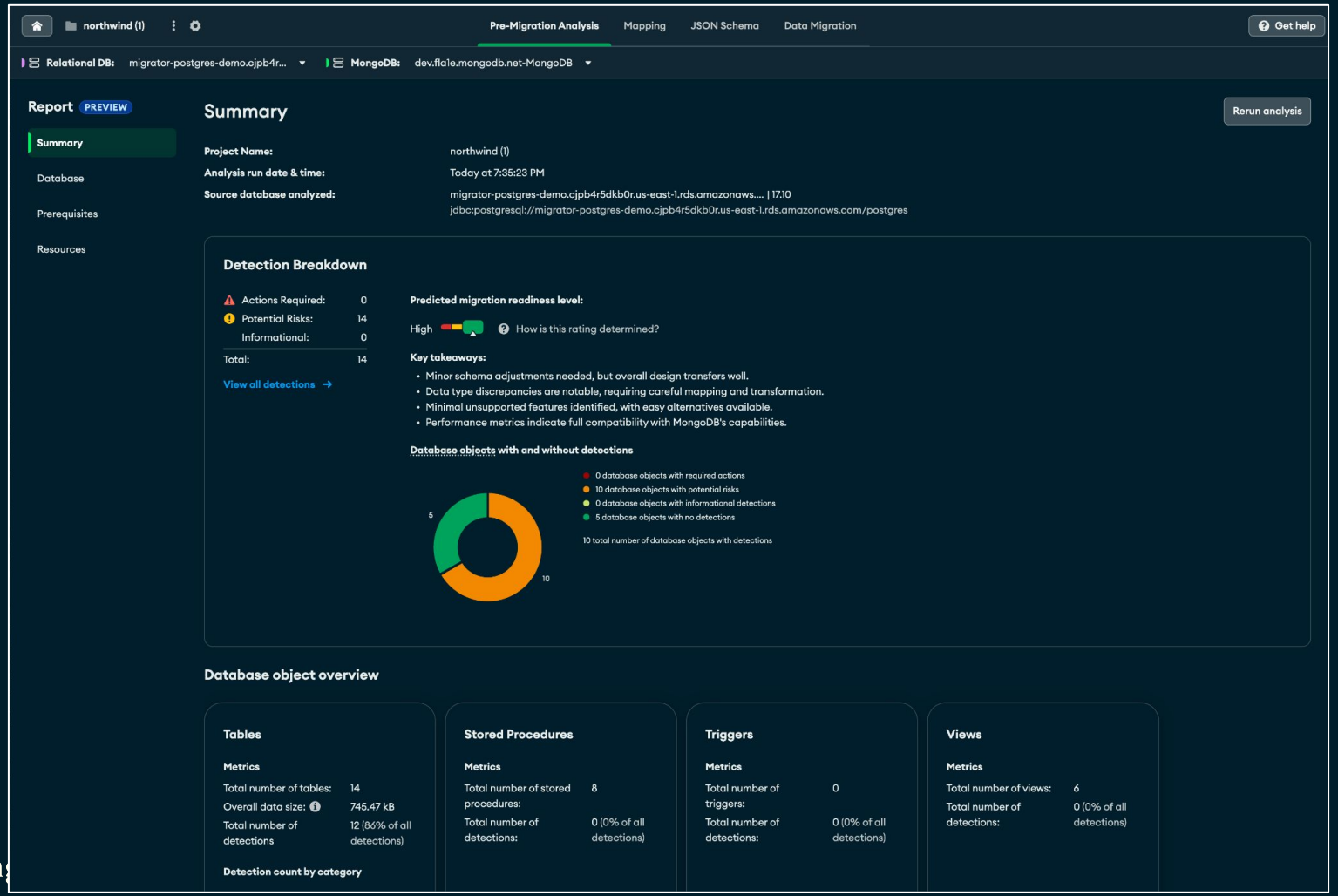
- Aurora_MySQL_to_Atlas_Migration_Guide.pdf 4단계 참고

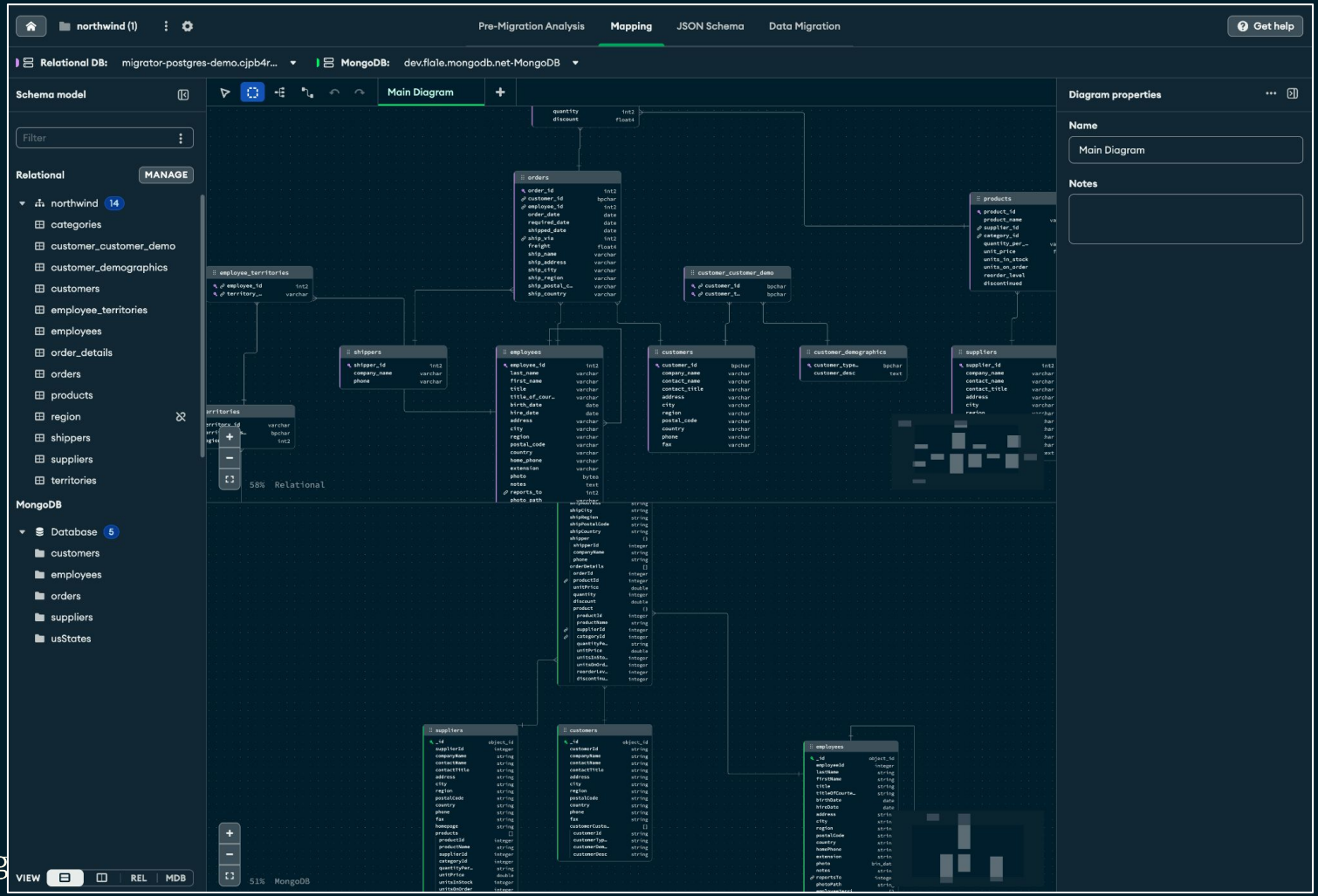
사용 중인 Migrator 설치 안내와 맞는 Connector/J JAR 사용

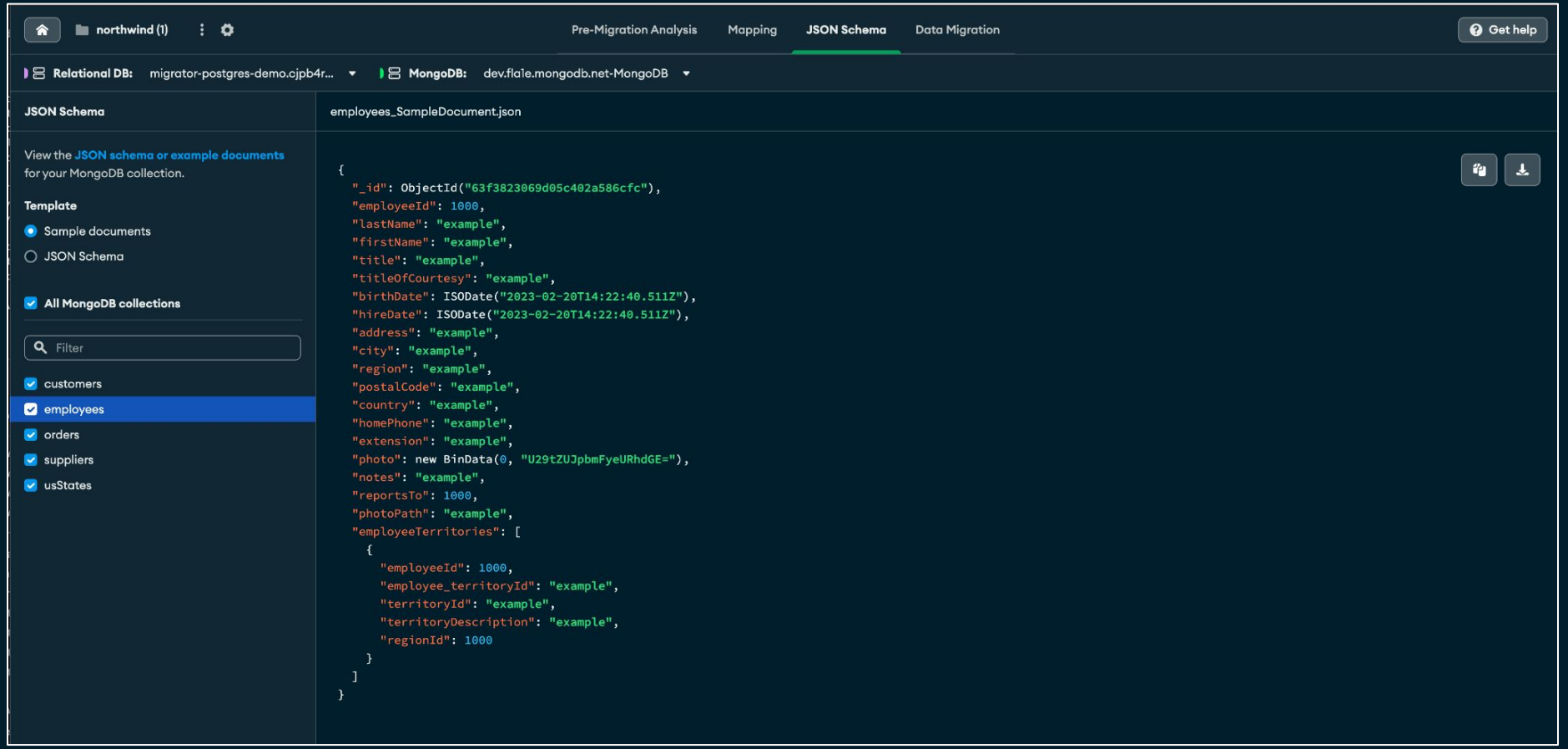
macOS: /Library/Application Support/MongoDB/Relational Migrator/Drivers

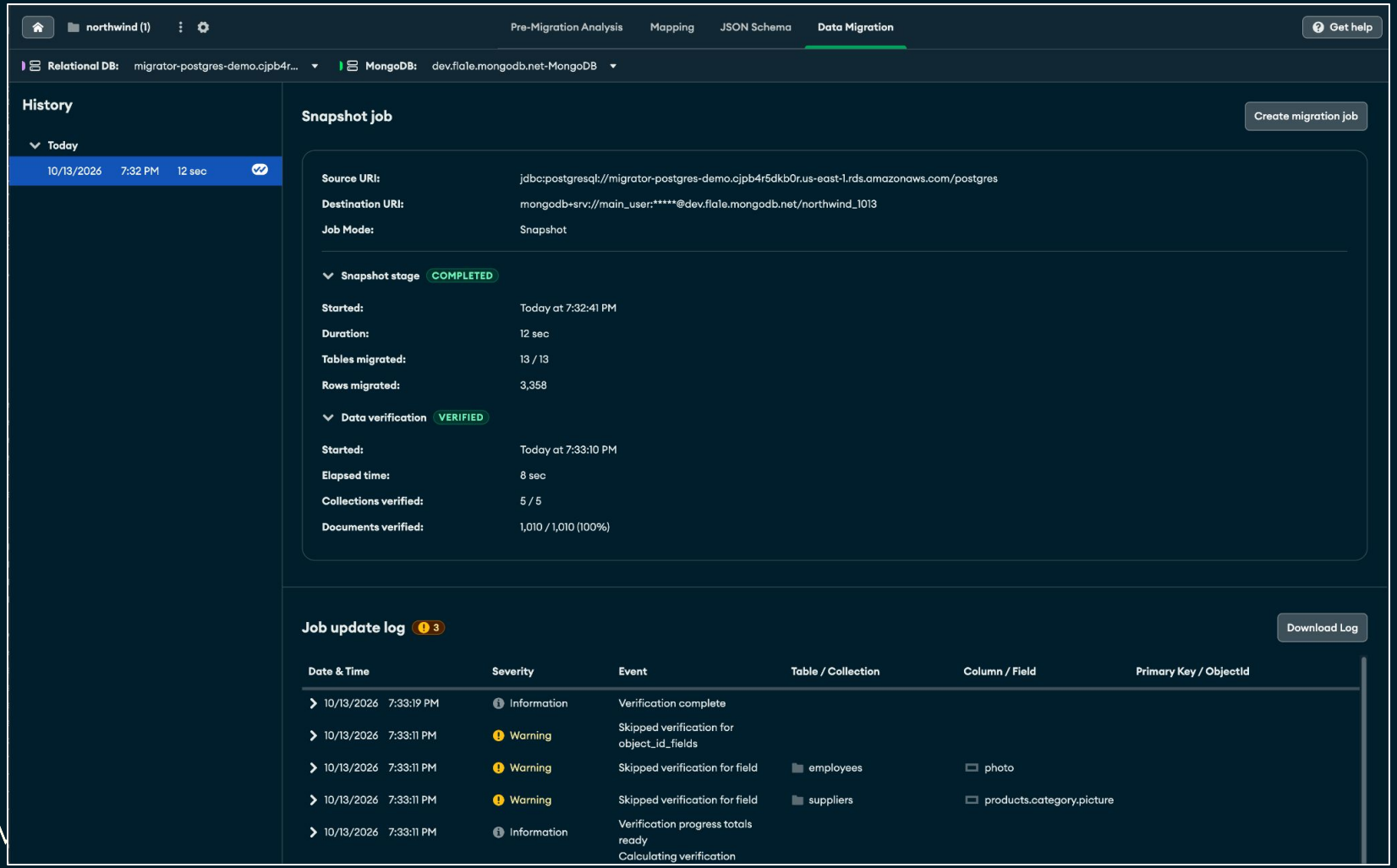
Windows: 사용자 AppData/Local/MongoDB/Relational Migrator/Data/Drivers

JAR 복사 후 Migrator 완전 종료·재실행 → 두 DB Test connection


northwind (1)
Pre-Migration Analysis
Mapping
JSON Schema
Data Migration
Get help
Relational DB: migrator-postgres-demo.cjpb4r...
MongoDB: dev.fla1e.mongodb.net-MongoDB
Report PREVIEW
Summary
Database
Prerequisites
Resources
Summary
Rerun analysis
Project Name: northwind (1)
Analysis run date & time: Today at 7:35:23 PM
Source database analyzed: migrator-postgres-demo.cjpb4r5dkb0r.us-east-1.rds.amazonaws.... | 17.10
jdbc:postgresql://migrator-postgres-demo.cjpb4r5dkb0r.us-east-1.rds.amazonaws.com/postgres
Detection Breakdown
Actions Required: 0
Potential Risks: 14
Informational: 0
Total: 14
View all detections
Predicted migration readiness level:
High
How is this rating determined?
Key takeaways:
Minor schema adjustments needed, but overall design transfers well.
Data type discrepancies are notable, requiring careful mapping and transformation.
Minimal unsupported features identified, with easy alternatives available.
Performance metrics indicate full compatibility with MongoDB's capabilities.
Database objects with and without detections
0 database objects with required actions
10 database objects with potential risks
0 database objects with informational detections
5 database objects with no detections
10 total number of database objects with detections
5
10
Database object overview
Tables
Metrics
Total number of tables: 14
Overall data size: 745.47 kB
Total number of detections 12 (86% of all detections)
Detection count by category
Stored Procedures
Metrics
Total number of stored procedures: 8
Total number of detections: 0 (0% of all detections)
Triggers
Metrics
Total number of triggers: 0
Total number of detections: 0 (0% of all detections)
Views
Metrics
Total number of views: 6
Total number of detections: 0 (0% of all detections)


northwind (1)
Pre-Migration Analysis
Mapping
JSON Schema
Data Migration
Get help
Relational DB: migrator-postgres-demo.cjpb4r...
MongoDB: dev.fla1e.mongodb.net-MongoDB
Schema model
Filter
Relational
MANAGE
northwind 14
categories
customer_customer_demo
customer_demographics
customers
employee_territories
employees
order_details
orders
products
region
shippers
suppliers
territories
MongoDB
Database 5
customers
employees
orders
suppliers
usStates
VIEW
REL
MDB
Main Diagram
58% Relational
51% MongoDB
Diagram properties
Name
Main Diagram
Notes


northwind (1)
Pre-Migration Analysis
Mapping
JSON Schema
Data Migration
Get help
Relational DB: migrator-postgres-demo.cjpb4r...
MongoDB: dev.fla1e.mongodb.net-MongoDB
JSON Schema
employees_SampleDocument.json
View the JSON schema or example documents for your MongoDB collection.
Template
Sample documents
JSON Schema
All MongoDB collections
Filter
customers
employees
orders
suppliers
usStates
{
"_id": ObjectId("63f3823069d05c402a586cfc"),
"employeeId": 1000,
"lastName": "example",
"firstName": "example",
"title": "example",
"titleOfCourtesy": "example",
"birthDate": ISODate("2023-02-20T14:22:40.511Z"),
"hireDate": ISODate("2023-02-20T14:22:40.511Z"),
"address": "example",
"city": "example",
"region": "example",
"postalCode": "example",
"country": "example",
"homePhone": "example",
"extension": "example",
"photo": new BinData(0, "U29tZUJpbmFyeURhdGE="),
"notes": "example",
"reportsTo": 1000,
"photoPath": "example",
"employeeTerritories": [
{
"employeeId": 1000,
"employee_territoryId": "example",
"territoryId": "example",
"territoryDescription": "example",
"regionId": 1000
}
]
}


northwind (1)
Pre-Migration Analysis
Mapping
JSON Schema
Data Migration
Get help
Relational DB: migrator-postgres-demo.cjpb4r...
MongoDB: dev.fla1e.mongodb.net-MongoDB
History
Today
10/13/2026 7:32 PM 12 sec
Snapshot job
Create migration job
Source URI: jdbc:postgresql://migrator-postgres-demo.cjpb4r5dkb0r.us-east-1.rds.amazonaws.com/postgres
Destination URI: mongodb+srv://main_user:*****@dev.fla1e.mongodb.net/northwind_1013
Job Mode: Snapshot
Snapshot stage COMPLETED
Started: Today at 7:32:41 PM
Duration: 12 sec
Tables migrated: 13 / 13
Rows migrated: 3,358
Data verification VERIFIED
Started: Today at 7:33:10 PM
Elapsed time: 8 sec
Collections verified: 5 / 5
Documents verified: 1,010 / 1,010 (100%)
Job update log 3
Download Log
Date & Time
Severity
Event
Table / Collection
Column / Field
Primary Key / ObjectId
10/13/2026 7:33:19 PM Information Verification complete
10/13/2026 7:33:11 PM Warning Skipped verification for object_id_fields
10/13/2026 7:33:11 PM Warning Skipped verification for field employees photo
10/13/2026 7:33:11 PM Warning Skipped verification for field suppliers products.category.picture
10/13/2026 7:33:11 PM Information Verification progress totals ready
Calculating verification

Snapshot으로 한 범위를 먼저 이관합니다

고객용 Aurora 이관 가이드와 동일한 흐름: hrm_migration_v1 → 검증 → hrm_pov 모델

01
범위·연결
고정 표본 + PK

→

02
매핑
원본 키·타입 보존

→

03
Snapshot
Job 실행·검증

→

04
대조
동일 키 값 확인

실습: 설치 버전·JDBC 안내 확인 → Table Filter → 새 목적지 DB → Job ID·생략 범위 기록

현재 상태 Snapshot은 과거 Notification 이력을 생성하지 않습니다

적재 완료는 건수·키·타입·값으로 확인합니다

VERIFIED 상태와 함께 검증 생략 범위까지 읽습니다.

확인	대조 방식	결과 기록
건수·범위	동일 Table Filter SQL ↔ countDocuments	원본 ___ / 목적지 ___
복합키	A/B 전체 키 · duplicate key · 누락	충돌 ___ / 누락 ___
타입·값	UTC Date / 숫자 정밀도 / NULL / 직렬화 배열	샘플 ___건 / 불일치 ___
모델 변환	staging → hrm_pov · 매핑 함수 version	변환 규칙 ___ / reject ___

staging 이관 성공과 최적 문서 모델·서비스 성능 검증은 각각 증거를 남깁니다

03

Lab 1: HRM 데이터 모델링

60 MIN

devices, device_current_state, device_events 등 설계
Validation 및 인덱스 구성

이 시간의 결과물은 세 가지입니다

설계한 문서 → 타입 확인 → 인덱스 생성

문서 4종

무엇을 한 문서에 담나?

devices / state / events / anomaly

Validation

필수 필드와 타입

Date · Object · Array · enum

인덱스 3개

어떻게 문서를 찾나?

식별 키 + 시간 정렬

고객 샘플에서 필요한 필드를 고릅니다

이벤트 식별 · 리소스 식별 · 시간 · payload · 기기 metadata

```
{
  deviceId: "<UUID>", eventId: "<UUID>",
  locationId: "<UUID>",
  data: { href: "/resourceA/vs/0" },
  component: "main",
  eventTime: "2026-01-01T00:00:00.000Z",
  timestamp: "2026-01-01T00:00:00.300Z",
  capabilities: "[execute]",        // string
  value: { payload: {
    modes: [ /* 6 items */ ],       // array
    supportedModes: [ /* 14 items */ ]
  } }
}
```

이벤트

eventId / 시각

최신 리소스

deviceId + unitId

타입

문자열과 배열 구분

샘플의 필드를 그룹화하되 원래 의미와 타입을 바꾸지 않습니다

추천 모델: 네 가지 컬렉션

작은 관련 객체는 Embed, 계속 쌓이는 이력은 별도 컬렉션

컬렉션	문서 한 개	포함할 정보
devices	기기 1대	product · versions · location
device_current_state	리소스 최신 상태 1개	Entity · key · state · updatedAt
device_events	Notification 1건	resource · 시간 · payload · context
anomaly_results	탐지 결과 1건	rule · status · detectedAt · evidence

devices.events[]처럼 전체 이력을 한 문서에 계속 추가하지 않습니다

devices: 기기 정보는 작은 객체로 묶습니다

기기 상세 조회는 _id 하나로

```
{
  _id: "D01",                    // deviceId
  locationId: "L01",
  product: {
    modelCode: "MODEL-A",
    deviceTypeId: "DeviceType-A"
  },
  versions: { firmwareVersion: "1.0" },
  location: { countryCode: "AAA" },
  schemaVersion: 1
}
```

Embed

product / versions

Reference

이력은 deviceId로 연결

Index

기본 _id만 사용

기본 시작안에는 **devices**의 추가 인덱스가 없습니다

device_current_state: 리소스별로 한 문서

Entity와 원래 복합키를 고정된 stateKey로 표현

```
{
  deviceId: "D01", entity: "A",
  key: { unitId: "0" },
  stateKey: '["A","0"]',
  state: {
    modes: ["MODE_01", "MODE_02"],
    supportedModes: ["SUPPORTED_MODE_01"]
  },
  updatedAt: ISODate("2026-10-12T08:00:00Z"),
  schemaVersion: 1
}
```

단위

기기 전체가 아닌 리소스

Embed

state 관련 필드

키 규칙

앱·이관에서 동일 함수

stateKey는 파생 키입니다.원래 키를 보존하고 쿼리에도 같은 규칙을 적용합니다

상위 쓰기에서 인덱스를 찾는 방법

SET 필드 수보다, 업데이트할 문서를 찾는 키가 먼저입니다

고객 상위 쓰기	원래 식별 키	우리 **filter**
A: 모드·옵션 **3,325.5 calls/s**	deviceId + unitId	deviceId + [A, unitId]
B: 필터 상태 **3,308.1 calls/s**	deviceId + unitId + component + filterName	deviceId + [B, unitId, component, filterName]
공통 인덱스	{ deviceId: 1, stateKey: 1 }	unique: true

모드를 수정한다고 **modes** 인덱스가 필요한 것은 아닙니다

device_events: 이벤트 한 건을 보존합니다

발생·수신 시각은 Date, payload 배열은 native array

```
{
  _id: "E01", deviceId: "D01",
  resource: { href: "/resourceA/vs/0",
              unitId: "0", component: "main" },
  eventTime: ISODate("2026-10-12T08:00:00Z"),
  receivedAt: ISODate("2026-10-12T08:00:00.300Z"),
  context: { modelCode: "MODEL-A" },
  notification: { capabilities: "[execute]" },
  payload: { modes: ["MODE_01"],
             supportedModes: ["SUPPORTED_MODE_01"] },
  schemaVersion: 1
}
```

별도 이력

이벤트 1건 = 문서 1개

Embed

resource / payload / context

Index

deviceId + eventTime

이벤트 당시 **context** 중복은 조회 목적과 보존 비용을 보고 결정합니다

anomaly_results: 탐지 결과도 독립 문서

오늘은 저장 형식과 인덱스를 준비하고, Lab 3에서 결과를 채웁니다

```
{
  deviceId: "D01",
  ruleId: "DEMO_DOOR_OPEN_180S",
  detectedAt: ISODate("2026-10-12T08:04:00Z"),
  status: "open",
  evidence: {
    durationSeconds: 240, thresholdSeconds: 180
  },
  sourceEventIds: [],
  exampleOnly: true, schemaVersion: 1
}
```

Embed

작은 evidence 객체

Reference

원본은 sourceEventIds

다음 세션

실제 탐지는 Lab 3

가상 결과 형식입니다. 실제 도어 코드 매핑은 후속 **Lab**에서 확인합니다

리소스별 문서와 기기별 문서를 비교합니다

시작안: device_current_state에 resource-level 문서. 비교안은 고객의 실제 read·write로 평가합니다.

모델 후보	장점 가설	PoV에서 측정
리소스별 통합 컬렉션	stateKey로 A–H 식별 · 독립 갱신	동일 key 경쟁 · 문서 크기
기기별 bounded 상태	기기 상세 1회 읽기 · 관련 상태 원자성	기기 hot key · 전체 문서 폭
Entity별 컬렉션	타입·보존·확장 정책 독립	조회 조합·인덱스·운영 수
events 별도 보존	재처리·기간 분석 · lifecycle 독립	이벤트/s · TTL · 저장량

쓰기 경쟁·문서 폭·독립 보존·파생 원자성으로 문서 경계를 선택합니다

누락 · null · 빈 배열의 의미부터 정합니다

Entity A payload는 제품마다 다릅니다. 냉장고 샘플에는 options · supportedOptions가 없습니다.

입력 형태	기본 제안 (고객 확인)	검증할 사례
필드 누락	변경 없음 · 기존 값 보존	modes만 도착 → options 보존
field: null	명시적 null 또는 삭제 계약	NULL 의미 / $unset 매핑
field: []	빈 배열로 명시적 갱신	활성 mode가 없는 상태
capabilities: "[execute]"	원문 보존 / 규칙 확인 후 별도 정규화	유효 JSON 배열 문자열은 아님

부분 갱신은 leaf 경로에 $set; state 전체 교체는 완전한 snapshot일 때 선택합니다

실습 1 · 내 문서와 식별 키를 적어봅니다

컬렉션	담을 객체·필드	찾을 키
devices	_____	_____
device_current_state	_____	_____
device_events	_____	_____
anomaly_results	_____	_____

같은 키의 관련 상태는 함께, 무한히 누적되는 이력은 따로

실습 2 · 기본 Validation을 설정합니다

필수 필드 · Date · Object · Array · enum부터

```
validator: { $jsonSchema: {
  bsonType: "object",
  required: ["deviceId", "eventTime", "payload"],
  properties: {
    deviceId: { bsonType: "string" },
    eventTime: { bsonType: "date" },
    payload: { bsonType: "object" }
  }
} }
// 신규 실습: strict / error
// 기존 이관 데이터: 점검 → warn → error 검토
```

required

누락 방지

type

문자열 날짜와 Date 구분

진화

추가 필드는 유연하게

6개·14개는 샘플 배열 길이이며, 실제 최대 길이 제약이 아닙니다

인덱스 원칙: 먼저 찾고, 그다음 정렬합니다

Equality → Sort/Range를 기준으로 쿼리와 연결

쿼리 목적	Equality	Sort/Range
리소스 최신 상태	deviceId + stateKey	없음
장치별 최근 이벤트	deviceId	eventTime DESC
장치별 최근 탐지 결과	deviceId	detectedAt DESC

인덱스를 많이 만드는 대신 자주 쓰는 쿼리 하나와 연결해서 만듭니다

실습 3 · 기본 인덱스 3개를 만듭니다

devices는 자동 생성된 _id 인덱스를 사용합니다

```
db.device_current_state.createIndex(
  { deviceId: 1, stateKey: 1 }, { unique: true }
);

db.device_events.createIndex(
  { deviceId: 1, eventTime: -1 }
);

db.anomaly_results.createIndex(
  { deviceId: 1, detectedAt: -1 }
);
```

상태

고유 리소스 찾기

이력

장치의 최근 이벤트

결과

장치의 최근 이상 탐지

운영 알림 목록·모델/지역 필터 인덱스는 실제 **reader** 확인 후 **Lab 2**에서 검토

실습 4 · 찾는 키와 바꾸는 값을 분리합니다

리소스 한 개만 찾고, 필요한 state 필드만 $set

```
db.device_current_state.updateOne(
  { deviceId: "D01", stateKey: '["A","0"]' },
  { $set: { "state.modes": ["MODE_01"] } }
);

db.device_events.find({ deviceId: "D01" })
  .sort({ eventTime: -1 })
  .limit(20);
```

filter

인덱스로 대상 문서 찾기

$set

변경 필드만 지정

읽기

시간 인덱스로 정렬

갱신 대상 필드가 곧 인덱스 필드는 아닙니다

추가 인덱스는 오후 실제 쿼리로 검증합니다

지금은 후보로만 남깁니다

필요할 때	후보 인덱스	확인할 것
미처리 알림 목록	status + detectedAt DESC	status 필터·정렬
리소스별 이력	deviceId + resource.href + eventTime DESC	기존 인덱스와 역할 비교
모델·지역별 기간 조회	context.modelCode + context.regionCode + eventTime	필터·선택도·쓰기 비용
배열 필터	실제 reader에 맞춰 선택	독립 두 배열 compound 금지

getIndexes는 지금, 상세 explain·집계 튜닝은 Lab 2에서

Lab 1 완료: 문서·타입·키·인덱스가 연결됐나요?

마지막 4분: 결정사항 정리와 다음 Lab 연결

확인	완료 기준	다음 세션
문서 4종	단위·Embed/Reference 설명	Lab 2에서 조회
리소스 키	A/B 전체 키 보존	Lab 3에서 순서·멱등 처리
Validation	필수·Date·Array·enum	실데이터로 확장
추가 인덱스 3개	getIndexes 확인	Lab 2에서 explain 검증

작게 시작하고, 실제 쿼리와 데이터로 필요한 규칙·인덱스만 더합니다

REFERENCE

부록 · Entity별 stateKey 규칙

API·Migrator·이벤트 처리기가 같은 규칙을 사용

Entity	stateKey 배열(deviceId 별도)	원천 근거
A	[A, unitId]	모드·옵션
B	[B, unitId, component, filterName]	필터 상태
C	[C, subDevice]	진단
D	[D, unitId]	코스·옵션
E	[E, unitId, id]	기기 정보
F	[F, unitId, target]	펌웨어·OTA

stateKey에는 modes·온도·timestamp처럼 바뀌는 값을 넣지 않습니다

04

Lab 2: Aggregation 과 인덱스

40 MIN

최근 상태, 문 열림 횟수·시간, 모델·지역별 집계 및 **explain()** 기반 인덱스 검증

Lab 2 데이터 복사 / 별도 실습 DB

```
# 컬렉션 복사
mongodump --uri="$URI" --db=sourceDb --collection=sourceColl --archive \
| mongorestore --uri="$URI" --archive \
  --nsFrom='sourceDb.sourceColl' --nsTo='targetDb.targetColl'

# DB 전체 복사
mongodump --uri="$URI" --db=sourceDb --archive \
| mongorestore --uri="$URI" --archive \
  --nsFrom='sourceDb.*' --nsTo='targetDb.*'
```

SQL의 filter와 SET을 MongoDB에 매핑합니다

선행 읽기를 줄이는 가설: 병합이 단순 부분 갱신이고, 변경·삭제 의미가 명확할 때

```
// 기존 A 문서 / W1 UPDATE
db.device_current_state.updateOne(
  { deviceId: "D01", stateKey: '["A","0"]' },
  { $set: { "state.modes": ["MODE_01"],
            updatedAt: new Date() } },
  { upsert: false }
);
// C/H W2: 업무 규칙 확인 후 upsert:true
// Kafka 배치 → updateOne 연산 목록 → bulkWrite
```

대상

원래 Entity·복합키

변경

도착한 필드만 $set

부하

갱신/파생 연산 수

$set은 변경 경로를 지정합니다. 내부 I/O·복제 비용 감소 폭은 직접 측정합니다

IN-list는 복합키 쌍을 그대로 조회합니다

A·D reader 조회 재현 → 선행 읽기의 이유 확인 → 필요 시 읽기 없는 갱신과 비교

```
// A의 (deviceId, unitId) 쌍을 보존
db.device_current_state.find({ $or: [
  { deviceId: "D01", stateKey: '["A","0"]' },
  { deviceId: "D02", stateKey: '["A","1"]' }
] }).explain("executionStats");

// 비교 시 batchSize·projection·readPreference 고정
// 인덱스: { deviceId: 1, stateKey: 1 } unique
// reader 호출률·배치 크기는 고객과 실측
```

조건

키 쌍 보존

검사

index / docs examined

비교

읽기+병합 vs 직접 갱신

deviceId IN × unitId IN은 원래 키 쌍의 조건과 다를 수 있습니다

실습 데이터: 이미 정규화한 가상 도어 이벤트

고객 MODE 코드를 임의로 OPEN/CLOSE로 해석하지 않습니다

```
{
  deviceId: "LAB2-D01",
  resource: { href: "/lab2/door/0", unitId: "0" },
  eventType: "door_close",       // 합성 의미 필드
  eventTime: ISODate("2026-10-11T10:04:00Z"),
  payload: {
    doorState: "closed",
    openedAt: ISODate("2026-10-11T10:00:00Z")
  },
  context: { modelCode: "LAB2-MODEL-1",
             regionCode: "DEMO-SEOUL" }
}
```

BSON Date

시간 계산 준비

openedAt

합성 정규화 필드

context

당시 모델·지역

기존 **Lab 1** 상태 데이터는 유지하고, 가상 도어 이력을 추가해 사용합니다

Aggregation은 문서를 단계별로 처리합니다

필요한 문서만 고르고 → 계산하고 → 묶고 → 정렬합니다

$match

대상·기간 필터

device / resource / time

$set + $group

계산과 집계

duration / count / average

$sort + $limit

정렬과 결과 제한

최신 이력 / 상위 결과

파이프라인 앞에서 입력을 줄이고,필요한 결과 모양으로 마무리합니다

최신 상태를 읽는 두 가지 방법

현재 상태 문서 조회와 이력의 마지막 관측 조회

```
// 이미 계산된 최신 상태: Lab 1 데이터
db.device_current_state.findOne({
  deviceId: "<Lab1 deviceId>", stateKey: '["A","0"]'
});

// 이력에서 마지막 도어 관측: Lab 2 데이터
db.device_events.aggregate([
  { $match: { deviceId: D,
      "resource.href": "/lab2/door/0" } },
  { $sort: { eventTime: -1 } },
  { $limit: 1 }
]);
```

직접 조회

current_state

마지막 관측

match → sort → limit

구분

실시간 현재 상태와 다름

resource 조건을 빼면 다른 종류의 **Notification**이 마지막 값으로 나올 수 있습니다

실습 1 · 날짜별 문 열림 횟수

OPEN만 세고, 현지 날짜로 묶습니다

```
db.device_events.aggregate([
  { $match: { deviceId: D, eventType: "door_open",
      "resource.href": "/lab2/door/0",
      eventTime: { $gte: START, $lt: END } } },
  { $group: {
      _id: { $dateToString: {
        date: "$eventTime", format: "%Y-%m-%d",
        timezone: "Asia/Seoul"
      } },
      openCount: { $sum: 1 }
  } },
  { $sort: { _id: 1 } }
]);
```

필터

OPEN + 기간

그룹

KST 날짜

정답

10/11 = 4회

시작 포함 · 끝 제외 : $gte START / $lt END

실습 2 · 열린 시간의 합계와 평균

완료 구간 20초·40초·240초 → 합계300초·평균100초

```
// CLOSE와 openedAt 기간을 먼저 $match
{ $set: { durationSeconds: {
    $divide: [
      { $subtract: ["$eventTime", "$payload.openedAt"] },
      1000
    ]
} } },
{ $group: {
    _id: null, closedCycles: { $sum: 1 },
    totalSeconds: { $sum: "$durationSeconds" },
    avgSeconds: { $avg: "$durationSeconds" },
    maxSeconds: { $max: "$durationSeconds" }
} }
```

OPEN

4회

완료

3구간

결과

300초 / 평균100초

열림 횟수의 분모와 평균 지속시간의 분모는 다릅니다

모델·지역별로 묶으면 비교 지표가 됩니다

기간 필터와 duration 계산은 유지하고 그룹 키를 변경

```
// 앞 단계: CLOSE 기간 필터 → durationSeconds 계산
{ $group: {
    _id: {
      model: "$context.modelCode",
      region: "$context.regionCode"
    },
    closedCycles: { $sum: 1 },
    totalSeconds: { $sum: "$durationSeconds" },
    avgSeconds: { $avg: "$durationSeconds" }
} },
{ $sort: { "_id.model": 1, "_id.region": 1 } }
```

그룹 키

모델 + 지역

분모

정상 완료 구간 수

결과

합성 데이터 6개 그룹

closedCycles는 완료 구간 수이며, 전체 OPEN 횟수가 아닙니다

실습 3 · 같은 파이프라인을 바꿔보세요

수정·실행 → 결과 공유

바꿀 것	수정 단계	확인할 것
디바이스를 **D02**로	$match	이력 범위가 달라졌나?
기간을 하루로	$match	UTC 경계를 맞췄나?
지역만으로 그룹	$group	분모는 무엇인가?
시간대별 **OPEN**	$hour + $group	timezone을 지정했나?

모든 쿼리를 새로 만들기보다 필터·계산·그룹 키를 바꾸어 재사용합니다

인덱스 가설: 장치뿐 아니라 리소스도 먼저 좁힙니다

이번 비교 대상은 '최근 리소스 이력20건' 조회입니다

Lab 1 인덱스

- deviceId → eventTime
- 장치·기간으로 탐색
- 다른 resource를 읽고 제외 가능

Lab 2 후보

- deviceId → resource.href → eventTime
- 두 equality 조건으로 먼저 탐색
- 같은20건에 필요한 검사량 비교

필터: deviceId + resource.href / 정렬·범위: eventTime

explain은 시간보다 먼저 '한 일'을 봅니다

같은 결과를 반환하면서 덜 읽고 덜 정렬했는가?

항목	뜻	이번 find 쿼리에서 확인
nReturned	반환한 문서 수	모든 경로20건
totalDocsExamined	검사한 문서 수	불필요한 문서가 줄었나?
totalKeysExamined	검사한 인덱스 키 수	조건으로 충분히 좁혔나?
IXSCAN / COLLSCAN	인덱스 / 전체 스캔	어떤 접근 경로인가?
SORT / FETCH	별도 정렬 / 문서 읽기	정렬 제거? FETCH는 정상 가능

0ms·1ms만으로 운영 성능이 몇 배 빨라졌다고 말하지 않습니다

실습 4 · 같은 쿼리로 경로를 비교합니다

기존 인덱스를 삭제하지 않고 새 후보1개를 만듭니다

```
db.device_events.createIndex(
  { deviceId: 1, "resource.href": 1, eventTime: -1 },
  { name: "lab2_device_resource_time" }
);

db.device_events.find(q)
  .sort({ eventTime: -1 }).limit(20)
  .hint("lab2_device_resource_time")
  .explain("executionStats");

// 비교: $natural / 기존 index / 새 index
// 마지막: hint 없이 선택된 계획 확인
```

같은 쿼리

filter / sort / limit

한 개 추가

candidate index

기존 유지

drop·hide 하지 않기

hint는 비교 실험 도구입니다. 운영 쿼리에 무조건 고정하는 결론은 아닙니다

기존 교육 예시: 같은 20건, 검사량 비교

기존 자료의 MongoDB 8.0.13 합성 예시 / 고객 운영 성능은 별도 측정

경로	검사 키	검사 문서	SORT
강제 전체 스캔(참고)	0	4,977	있음
기존 Lab 1 인덱스	77	77	없음
새 복합 인덱스	20	20	없음
hint 없는 선택	20	20	없음

기존 인덱스도 정렬은 지원합니다. 새 후보의 차이는 리소스 조건의 선택도입니다

이 네 가지만 주의하면 집계가 정확해집니다

지표의 의미와 쿼리의 실제 입력을 확인하세요

실수	바꿀 방법	확인
전체 이벤트 수 ÷ 2	OPEN 이벤트만 count	4회 vs 완료3구간
미완료 구간을0초로	정상 완료 구간만 평균	분모를 별도 표시
UTC 날짜로 무조건 그룹	업무 timezone 지정	시작 포함·끝 제외
한 인덱스로 모든 집계 최적화	실제 쿼리별 explain	group 반환수 해석 주의

계산식과 분모가 맞아야 빠른 쿼리도 의미가 있습니다

Lab 2 완료: 쿼리와 인덱스를 설명할 수 있나요?

결과 정리와 Lab 3 연결

Aggregation

필터 → 계산 → 그룹

횟수·시간·모델·지역

인덱스

같은 결과, 적은 검사

기존 → 후보 → hint 없는 선택

다음 Lab

이벤트 품질

원천 매핑·openedAt·지연·알림

05

Lab 3: 이벤트 품질 및 이상 감지

40 MIN

Kafka 이벤트 적재, **Data Federation,** 멱등 처리
중복·지연·순서 역전 처리 및 **3분** 초과 문 열림 탐지

이벤트가 중복·지연되어도 상태는 정확해야 합니다

5분 개념 → 10분 적재·재전송 → 10분 순서·부분 갱신 → 10분 이상 감지 → 5분 결과 정리

01
Notification
eventId + 원문 보존

→

02
적재·정규화
Date · 복합키 · payload

→

03
상태·감지
조건부 갱신 + timer

→

04
결과 확인
재전송·지연 시나리오

목표: 재전송·지연·재시작 후에도 이벤트 수·최신 상태·감지 결과가 동일함을 확인합니다

CONCEPT

이벤트 품질 문제 세 가지와 MongoDB 해법

Kafka는 최소 1회(at-least-once) 전달이 일반적입니다. 저장 단계에서 결과를 안정적으로 만듭니다.

중복 전송

같은 이벤트가 두 번

eventId → _id
두 번 저장해도 이력 1건

지연·역전

오래된 값이 늦게 도착

발생 시각/버전 비교
최신 값만 반영

미도착

“닫힘”이 오지 않음

주기 timer 평가
열린 시간 계산

핵심: 같은 입력을 몇 번 처리해도 같은 결과(멱등) + 오래된 값이 최신 값을 덮지 않음(최신성)

Kafka 적재와 S3 이력 조회의 역할을 나눕니다

Kafka topic(또는 동일 JSON replay) → Atlas 전용 DB. 과거 이력은 S3에 둔 채 함께 조회합니다.

구성 요소	역할	실습에서 확인
Kafka → Sink Connector	원본 이벤트 이력 적재	topic·partition·offset · 적재 지연·거부 건수
처리기 (consumer)	복합키 추출 · 부분 갱신 · 최신성 판정	재시작·중복 후 상태 정합성
S3 + Data Federation	과거 이력과 Atlas 데이터 통합 조회	같은 기기·기간 결과 일치
Atlas 컬렉션	current_state / events / anomaly	키·타입·인덱스·원천 추적

Data Federation은 S3·Atlas를 함께 조회하는 계층이고, 실시간 적재는 Kafka Connector/처리기가 담당합니다

같은 eventId를 다시 받아도 이력은 한 건입니다

eventId가 전역 유일하면 _id로 사용합니다. 아니라면 source + deviceId + eventId 조합을 _id로 만듭니다.

```
// e: 정규화된 이벤트 (_id 필드 제외)
db.device_events.updateOne(
  { _id: e.eventId },
  { $setOnInsert: e },
  { upsert: true }
);

// 실습: 같은 e를 두 번 실행 → 이력 1건
// upsertedCount 1 → 0 변화를 확인
```

이력

결정적 _id

업무 중복

별도 규칙

재시작

상태 갱신 재적용

이력이 한 건인지와 함께, 최신 상태·파생 결과까지 동일한지 확인합니다

늦게 도착한 이벤트가 최신 상태를 되돌리지 않게 합니다

수신 시각이 아닌 발생 시각(eventTime) 또는 원천 순번으로 최신성을 판단합니다.

상황	처리 방법	실습 확인
전체 상태 (snapshot)	더 새로운 eventTime일 때만 전체 반영	T2 반영 후 T1 도착 → T2 유지
일부 필드 (patch)	필드별 시각/버전 비교 검토	다른 필드의 늦은 값 보존
같은 시각	원천 순번 · 동률 규칙 정의	동일 시각 상충 이벤트
동시 쓰기	조건부 update + matchedCount 확인	경쟁 시 결과·재시도

전체 상태(snapshot)인지 일부 필드(patch)인지에 따라 최신성 비교 단위를 다르게 설계합니다

조건부 갱신으로 최신 값만 반영합니다

상태 문서가 이미 존재하는 전체 상태(snapshot) 예제입니다. 신규 문서 생성은 별도로 처리합니다.

```
// e.eventTime: BSON Date
const r = db.device_current_state.updateOne(
  { deviceId: e.deviceId, stateKey: e.stateKey,
    $or: [ { observedAt: { $exists: false } },
           { observedAt: { $lt: e.eventTime } } ] },
  { $set: { state: e.payload,
            observedAt: e.eventTime,
            lastEventId: e.eventId } },
  { upsert: false }
); // matchedCount 0 → 오래된 이벤트
```

전제

전체 상태

신규 생성

키만으로 별도

검증

T2 → T1 역전

주의: 최신성 조건과 upsert:true를 함께 쓰면 오래된 이벤트가 중복 키 오류를 만들 수 있습니다

배치 쓰기와 파생 쓰기의 보장 범위를 확인합니다

한 이벤트가 상태(A)와 진단(C) 두 문서를 함께 갱신할 수 있습니다.

같은 키

순서를 보호

키별 순차 처리 또는
최신성 조건 갱신

ordered:false

독립 연산 병렬화

실패 건 별도 집계
순서 보장 가정 금지

A → C 파생

완료 기준 정의

트랜잭션 필요 여부
재처리 로그로 대조

bulkWrite는 배치 전체를 하나의 트랜잭션으로 묶지 않습니다. A·C 동시 반영 필요 여부를 정합니다

이벤트가 오지 않아도 열린 시간을 평가합니다

실습 규칙: 문이 180초 이상 열려 있으면 감지. 운영 임계값·알림 정책은 Samsung HRM 기준으로 확정합니다.

01
OPEN
openSince + ID 저장

→

02
timer
NOW − openSince

→

03
≥ 180초
아직 OPEN인지 확인

→

04
결과
구간당 1건 기록

CLOSE: 열린 시간 저장 · 반복 OPEN: 최초 시각 유지 · 늦은 CLOSE: 감지 결과 해소 처리

OPEN 시 openSince 기록 → 주기 timer 평가 → 같은 개방 구간은 감지 결과 한 건

고정된 NOW로 180초 경계를 확인합니다

실습용 도어 상태 문서(state.doorState, state.openSince)로 경계값을 검증합니다.

```
const NOW = ISODate("2026-10-11T10:03:00Z");
const cutoff = new Date(NOW.getTime() - 180000);
db.device_current_state.find({
  entity: "LAB_DOOR",
  "state.doorState": "open",
  "state.openSince": { $lte: cutoff }
});
// 결과: 열림 구간 ID로 한 번만 기록
// 기록 전 최신 상태 재확인
```

준비

도어 상태 문서

경계

179 / 180 / 181초

알림

구간당 1건

NOW 기준 179초는 제외, 180·181초는 감지 대상입니다

Lab 3 쿼리를 받쳐주는 인덱스

모든 필드에 인덱스를 만들지 않고, 자주 실행되는 조회·갱신 조건에만 만듭니다.

쿼리 / 목적	인덱스	비고
이벤트 중복 방지	_id = eventId (기본 인덱스)	추가 인덱스 불필요
최신 상태 조건부 갱신	{ deviceId: 1, stateKey: 1 } unique	Lab 1에서 생성
기기별 최근 이벤트	{ deviceId: 1, eventTime: -1 }	범위·정렬 동시 지원
열린 문 timer 평가	{ "state.openSince": 1 } partial (open)	열린 문서만 색인
감지 결과 1건 보장	{ deviceId, ruleId, openEventId } unique	알림 중복 방지

생성 후 explain("executionStats")와 Atlas Performance Advisor로 실제 사용 여부를 확인합니다

시나리오별 기대 결과와 실제 결과를 기록합니다

정상 · 중복 · 지연 · 역전 · 재시작 이후 재처리를 차례로 주입합니다.

주입 시나리오	기대 결과	실제 결과
같은 eventId 2회	이력 1건 · 상태·파생 결과 동일	
T2 반영 후 T1 도착	최신 상태 T2 유지	
modes만 포함된 patch	다른 필드 값 유지	
180초 경계 · 열린 상태 유지	timer 감지 · 구간당 결과 1건	
이력 저장 후 처리기 재시작	상태·파생 결과 재처리 완료	

공통 확인 항목: 입력 건수 / 고유 이벤트 / 최신 상태 / 파생 결과 / 거부·재시도 / 지연

06

Lab 4: Time Series Discovery

30 MIN

Time Series Collection 생성, 이벤트 적재·집계, **TTL** 적용 및 일반 컬렉션과 비교

연속 측정 데이터는 Time Series로 비교합니다

5분 개념 → 8분 생성·적재 → 8분 집계·저장 비교 → 4분 TTL → 5분 적용 여부 결정

일반 컬렉션

최신 상태 · 원본 이벤트

복합키 갱신·upsert
unique · Validation

Time Series

연속 측정 · 기간 집계

시간 + 고정 메타데이터
저장 효율 · 집계

판단 기준

같은 데이터로 비교

집계 결과 일치
저장량 · 조회 · 적재

최신 상태는 일반 컬렉션, 온도·전력처럼 계속 쌓이는 측정값은 Time Series 후보입니다

Time Series 설계: 세 가지 설정이 핵심입니다

HRM Notification 구조에 대응한 설정 예입니다. 컬렉션 생성 후 timeField·metaField는 바꿀 수 없습니다.

설정	HRM 매핑 예	팁
timeField	eventTime (발생 시각, BSON Date)	수신 시각과 구분 · 범위 조건에 사용
metaField	{ deviceId, resource, modelCode }	고정 식별값만 · eventId·mode 제외
granularity	seconds / minutes / hours	실제 측정 간격에 맞춤
보존 (TTL)	expireAfterSeconds	보존 기간 경과 bucket 자동 삭제
적재	Kafka Sink Time Series 설정	원본 이벤트 컬렉션과 병행 가능

metaField에는 잘 바뀌지 않는 식별값만, 측정값과 가변 상태는 문서 본문에 둡니다

Time Series 컬렉션을 만들고 측정값을 적재합니다

실습용 온도 측정값(tempC)을 사용합니다.

```
db.createCollection("measurements_ts", {
  timeseries: { timeField: "eventTime",
    metaField: "meta", granularity: "seconds" }
});
db.measurements_ts.insertOne({
  eventTime: new Date(),
  meta: { deviceId: "LAB4-D01",
          resource: "/lab4/temperature/0" },
  tempC: 3.5
});
```

시간

BSON Date

meta

기기 + 측정 리소스

간격

측정 주기에 맞춤

meta에는 시계열을 구분하는 고정 값만 넣습니다

같은 기간을 두 컬렉션에서 집계해 비교합니다

measurements(일반)와 measurements_ts에 같은 측정값 20 · 40 · 60을 넣고 비교합니다.

```
db.measurements_ts.aggregate([
  { $match: { "meta.deviceId": "LAB4-D01",
      eventTime: { $gte: START, $lt: END } } },
  { $group: { _id: { $dateTrunc: {
      date: "$eventTime", unit: "hour",
      timezone: "Asia/Seoul" } },
      count: { $sum: 1 }, avg: { $avg: "$tempC" },
      min: { $min: "$tempC" }, max: { $max: "$tempC" } } }
]);
```

필터

meta + 시간 범위

단위

시간 bucket

대조

일반 컬렉션 동일

기대 결과: count 3 / avg 40 / min 20 / max 60 — 두 컬렉션 결과가 같아야 합니다

보존 기간은 expireAfterSeconds로 자동 관리합니다

저장량 비교와 분리해, 별도 실습 컬렉션에서 만료 동작을 관찰합니다.

```
db.createCollection("measurements_ttl_demo", {
  timeseries: { timeField: "eventTime",
    metaField: "meta", granularity: "seconds" },
  expireAfterSeconds: 60
});
db.measurements_ttl_demo.insertOne({
  eventTime: new Date(Date.now() - 7200000),
  meta: { deviceId: "TTL-DEMO" }, tempC: 3.5
});
// 주기적으로 countDocuments({}) 확인
```

생성

실습 전용 컬렉션

관측

백그라운드 삭제

운영

업무 기준 보존

TTL 삭제는 백그라운드에서 bucket 단위로 진행되어, 만료 직후 즉시 사라지지 않을 수 있습니다

같은 측정 데이터로 정확도와 비용을 비교합니다

데이터 양 · 기간 · 클러스터 · 인덱스 조건을 동일하게 맞춘 뒤 측정합니다.

지표	일반 컬렉션	Time Series	확인 방법
집계 결과 (count/avg/min/max)			값 일치 여부
저장 + 인덱스 크기			collStats / 인덱스 크기
조회 지연 p50 / p95			동일 질의 반복 실행
적재 처리량 · 지연			동일 배치 입력
늦은 도착 · 재처리			중복 발생 여부

Time Series explain은 bucket 단위로 읽으므로, 일반 컬렉션의 문서 검사 수와 단위가 다릅니다

Time Series를 쓰기 전에 확인할 제약

MongoDB 8.0 기준입니다. 사용하는 클러스터 버전의 문서로 다시 확인합니다.

기능	일반 컬렉션	Time Series
최신 상태 upsert	지원	upsert 미지원 · 갱신은 metaField 기준 제한
unique 인덱스 (eventId 중복 방지)	지원	미지원 → 원본 단계에서 중복 처리
Schema Validation · Change Streams	지원	미지원
트랜잭션 내 쓰기	지원	미지원 (읽기는 가능)

최신 상태 · 중복 방지 · 스키마 검증은 일반 컬렉션에서, 기간 측정 분석은 Time Series에서 처리합니다

Time Series 적용 여부를 비교 결과로 결정합니다

HRM 이벤트 중 온도 · 전력처럼 일정 간격으로 쌓이는 측정값을 대상으로 판단합니다.

대상 데이터

측정 이벤트 선정

측정 단위 · 주기
timeField · metaField

비교 결과

일반 컬렉션 대비

집계 일치
저장 · 성능 · 재처리

결정

포함 / 보류 / 제외

결정 이유
추가 데이터 · 담당

측정 대상 · 주기 · 보존 기간이 정해지면 정식 PoV 검증 항목으로 등록합니다

07

Lab 5: Vector Search/RAG Discovery

35 MIN

매뉴얼 임베딩 적재, **Vector Search, Metadata Filtering** 및 근거가 포함된 답변 생성

증상 설명으로 매뉴얼의 근거를 찾습니다

5분 검색 유형 → 5분 필드·인덱스 → 10분 벡터 검색 → 7분 하이브리드·리랭킹 → 5분 RAG → 3분 평가

01
질문·상태
증상 + 모델·국가

→

02
검색
벡터 + 키워드 + 필터

→

03
리랭킹
상위 결과 재정렬

→

04
답변
출처 포함 생성

목표: 사용자의 표현이 매뉴얼 문구와 달라도 올바른 근거 문서를 찾고, 출처와 함께 답합니다

어떤 검색에 Vector Search가 필요한가요?

질의 유형에 따라 검색 방식을 고르고, 실제 서비스에서는 함께 조합합니다.

질의 예시	유형	권장 방식
모델 코드 · 국가 · 문서 유형 일치	정확한 값	일반 인덱스 / Vector filter
"<오류코드> 조치 방법"	키워드 · 코드	Atlas Search (한국어 분석기)
"문 아래에 물이 고여요"	증상 서술 · 표현 차이	Vector Search
"<오류코드> 뜨고 냉기가 약해요"	코드 + 서술 혼합	Hybrid ($rankFusion)
상위 결과 순서를 더 정확하게	정밀 재정렬	Native Reranking ($rerank)

정확한 값은 필터·키워드로, 의미·표현 차이는 벡터로, 둘이 섞인 질의는 하이브리드 + 리랭킹으로

매뉴얼 chunk 문서의 추천 필드와 인덱스 유형

manual_chunks 컬렉션 예입니다. 필터 값은 기기 메타데이터(modelCode · countryCode 등)와 같은 체계를 씁니다.

필드	용도	인덱스 유형
text (제목 > 섹션 > 본문)	의미 검색 · 키워드 검색 대상	vector (또는 autoEmbed) + Search(nori)
errorCode	오류 코드 정확 일치	filter + Search(token)
modelCode · deviceTypeId	제품 범위 제한	filter
country · language · docType	국가 · 언어 · 문서 유형	filter
publishedAt · firmwareVersion	최신 문서 · 버전 범위	filter (범위 조건)
source.uri · page · chunkId	답변 인용 · 출처	저장만 (인덱스 불필요)
embeddingModel · chunkVersion	재임베딩 관리	저장 (필요 시 filter)

벡터는 본문 1개 필드에, 필터는 거의 모든 질의에 들어가는 조건에만 지정합니다

chunk 하나에 본문·범위·출처를 함께 저장합니다

실습용 문서 형식입니다. 실제 매뉴얼의 경로와 페이지를 source에 기록합니다.

```
{
  _id: "MANUAL-01:p12:c01",
  modelCode: "MODEL-CODE-A", country: "KR",
  language: "ko", docType: "MANUAL",
  title: "배수 · 물고임", errorCode: null,
  text: "배수 · 물고임 > 점검 > <본문>",
  source: { uri: "<매뉴얼 경로>", page: 12 },
  embedding: [ /* D차원 */ ],
  embeddingModel: "<모델명>", chunkVersion: 1
}
```

범위

모델·국가·문서 유형

출처

uri + page

벡터

모델 출력 D차원

문서와 질문은 같은 임베딩 모델 · 같은 차원 · 같은 전처리로 만듭니다

쿼리 분석 결과로 인덱스와 필터를 정합니다

실제 질의 패턴을 먼저 보고, 자주 쓰는 조건만 인덱스에 반영합니다.

단계	도구	확인할 내용
① 질의 패턴 파악	Query Profiler · Performance Advisor · $queryStats	자주 쓰는 조건 조합 · 느린 질의
② pre-filter 지정	Vector index type: "filter"	거의 항상 쓰는 등호 · 범위 · $in
③ post-filter	$vectorSearch → $match	가끔 쓰는 복잡한 조건
④ 한국어 분석기	Search 인덱스 lucene.nori / lucene.korean	토큰 분리 결과 비교
⑤ 실행 확인	explain("executionStats") · vectorTracing	누락 문서 원인 · 단계별 시간

측정 → 인덱스 설정 → explain으로 확인 → 다시 측정의 순서로 반복합니다

벡터 설정은 모델과 측정 결과로 정합니다

처음에는 권장 기본값으로 시작하고, 정답 질문셋으로 정확도와 지연을 측정해 조정합니다.

설정	시작 권장값	조정 기준
numDimensions	임베딩 모델 출력 차원과 동일	변경 시 인덱스 재생성
similarity	모델 권장 (정규화 시 dotProduct)	모델 문서 확인
quantization	none → 대규모 시 scalar / binary	메모리 절감 vs recall
numCandidates	limit × 20 (예: limit 5 → 100)	recall 부족 · 좁은 필터 시 증가
정확도 측정	exact: true (ENN) 결과와 비교	ANN 겹침 90~95% 수준

ANN 결과를 exact:true(ENN) 결과와 비교해 recall을 측정하고 numCandidates를 조정합니다

벡터와 필터를 하나의 Vector Search 인덱스에 정의합니다

임베딩을 직접 생성해 저장하는 방식(BYO)의 예입니다. D는 실제 모델 차원으로 바꿉니다.

```
// name: manual_vector_index (type: vectorSearch)
{ fields: [
  { type: "vector", path: "embedding",
    numDimensions: D, similarity: "cosine" },
  { type: "filter", path: "modelCode" },
  { type: "filter", path: "country" },
  { type: "filter", path: "docType" },
  { type: "filter", path: "publishedAt" }
] }
```

벡터

모델 출력 차원

필터

모델·국가·유형·일자

상태

READY 확인

일반 B-tree 인덱스와 Vector Search 인덱스는 생성 방법과 상태 확인 방법이 다릅니다

임베딩 생성까지 Atlas에 맡길 수 있습니다

autoEmbed 인덱스는 저장·변경되는 텍스트와 질의 텍스트의 임베딩을 자동 생성합니다 (Voyage AI 모델).

```
// 인덱스 (type: vectorSearch)
{ fields: [
  { type: "autoEmbed", modality: "text",
    path: "text", model: "voyage-4" },
  { type: "filter", path: "modelCode" },
  { type: "filter", path: "country" } ] }

// 질의: 텍스트로 검색
{ $vectorSearch: { index: "manual_auto_index",
    path: "text", query: { text: "문 아래 물이 고여요" },
    numCandidates: 100, limit: 5 } }
```

모델

voyage-4 계열

요건

MongoDB 8.3+

질의

query.text

질의에 벡터 대신 텍스트를 넣습니다 — 임베딩 파이프라인 없이 시작해 정확도를 먼저 검증할 수 있습니다

같은 질문으로 필터를 바꿔 결과를 비교합니다

queryVector는 문서와 같은 모델로 만든 질문 벡터입니다.

```
db.manual_chunks.aggregate([
  { $vectorSearch: { index: "manual_vector_index",
      path: "embedding", queryVector: qv,
      numCandidates: 100, limit: 5,
      filter: { modelCode: "MODEL-CODE-A",
                country: "KR", docType: "MANUAL" } } },
  { $project: { title: 1, text: 1, source: 1,
      score: { $meta: "vectorSearchScore" } } }
]);
```

실습 1

필터 정확성

실습 2

정답 Top-5 포함

실습 3

출처 원문 확인

점수(vectorSearchScore)는 의미 유사도입니다. 답변의 정확성은 출처 확인으로 판단합니다

오류 코드와 증상이 섞인 질문은 두 검색을 합칩니다

$rankFusion으로 벡터 검색과 한국어 키워드 검색의 순위를 합칩니다 (MongoDB 8.0+).

```
db.manual_chunks.aggregate([
 { $rankFusion: { input: { pipelines: {
   vector: [ { $vectorSearch: { index: "manual_vector_index",
     path: "embedding", queryVector: qv,
     numCandidates: 200, limit: 20,
     filter: { country: "KR" } } } ],
   keyword: [ { $search: { index: "manual_text_ko",
     text: { query: "<오류코드> 냉기 약함",
             path: ["title", "text"] } } },
     { $limit: 20 } ] } },
   combination: { weights: { vector: 1, keyword: 1 } } } },
 { $limit: 20 }
]);
```

벡터

증상 의미

키워드

코드 · 용어 (nori)

결합

순위 기반 융합

각 파이프라인에 같은 범위 조건을 적용하고, 가중치는 정답 질문셋으로 조정합니다

관리형 리랭킹으로 상위 결과의 순서를 다듬습니다

$rerank는 Voyage AI 리랭커가 질문과 문서를 함께 읽고 관련도 순으로 다시 정렬하는 집계 단계입니다.

1. 넓게 검색

$vectorSearch · $rankFusion

후보 20~50건
필터로 범위 제한

2. 리랭킹

$rerank (rerank-2.5)

질문 + 지시문으로
관련도 재정렬

3. 답변

상위 5건 → LLM

근거가 정확해져
잘못된 답변 감소

검색으로 넓게 가져오고(예: 50건) → $rerank로 정밀 재정렬 → 상위 5건만 LLM에 전달합니다

벡터 검색 결과 뒤에 $rerank를 붙입니다

같은 질문셋으로 리랭킹 적용 전·후의 정답 순위를 비교합니다.

```
db.manual_chunks.aggregate([
  { $vectorSearch: { index: "manual_vector_index",
      path: "embedding", queryVector: qv,
      numCandidates: 400, limit: 50,
      filter: { modelCode: "MODEL-CODE-A" } } },
  { $set: { vsScore: { $meta: "vectorSearchScore" } } },
  { $rerank: {
      query: { text: "문 아래 물고임 · 배수 원인 우선" },
      model: "rerank-2.5", numDocsToRerank: 50,
      path: ["title", "text"] } },
  { $limit: 5 },
  { $project: { title: 1, source: 1, vsScore: 1,
      score: { $meta: "score" } } }
]);
```

입력

상위 50건

모델

rerank-2.5

출력

상위 5건 + 점수

$rerank는 $meta "score"를 리랭크 점수로 바꿉니다. 이전 점수가 필요하면 미리 다른 필드에 저장합니다

리랭킹 사용 전 확인할 조건

최신 기능이므로 사용하는 Atlas 클러스터 버전과 프로젝트 설정을 먼저 확인합니다.

항목	내용
배포 환경	Atlas 전용 (자체 관리형 · Atlas Local 미지원)
버전 · 설정	MongoDB 9.0+ 권장 · Project Settings에서 활성화 (Project Owner)
모델	rerank-2.5 / 2.5-lite (다국어 · 지시 반영) · rerank-3 계열 Preview
입력 제한	numDocsToRerank ≤ 1000 · path 필드 필수 존재
사용 위치	$vectorSearch · $search · $rankFusion 뒤 (fusion 입력 내부 불가)
운영	토큰 사용량 · Rate Limits · 지연 모니터링 / 비용 발생

Project Settings에서 Native Reranking을 켜고, Usage · Rate Limits 화면에서 사용량을 관리합니다

답변의 각 문장을 근거 문서와 연결합니다

검색 결과(근거)와 기기 최신 상태(관측값)를 구분해 LLM에 전달합니다.

입력

질문 + 상태 + 근거

질문 · 상태 관측 시각
chunkId · source · page

답변

요약 + 점검 절차

근거로 확인된 내용만
문장별 출처 표시

검증

원문과 대조

관련 문서인가?
주장이 출처에 있는가?

근거가 없으면 추측하지 않고, 필요한 정보(모델 · 오류 코드 · 상태 시각)를 다시 요청하도록 합니다

검색 품질과 답변 품질을 나누어 평가합니다

대표 질문과 정답 문서를 미리 정의하고, 리랭킹 전·후를 함께 비교합니다.

평가 항목	확인 방법	결과
검색 정확도	정답 chunk Recall@5 (벡터 / 하이브리드)	
리랭킹 효과	$rerank 전·후 정답 순위 · Recall@5	
필터 정확성	다른 모델 · 국가 · 문서 유형 혼입 건수	
답변 근거	문장 ↔ 출처 연결률 · 문서 외 추측 여부	
업무 유용성 · 성능	평가자 5점 척도 · 검색/생성 p50 · p95	

적용 여부는 검색 정확도 · 필터 · 근거 · 업무 유용성 · 응답 시간의 결과로 결정합니다

08

결과 리뷰 및 PoV 반영

20 MIN 성공 기준·검증 범위·Owner·일정 합의

성공 기준은 업무 가치와 측정 조건을 묶습니다

아래는 합의용 초안입니다. 기존 Must 기준이 있으면 우선 반영합니다.

트랙	성공 기준 초안	측정 / 목표 합의
Must · 데이터	선정 범위 키·타입·대표 값 일치	불일치 0건 제안
Must · 워크로드	A 중심 갱신·R1·최근 이력 정상 실행	부하·p95·freshness ___
Must · 품질	replay·역전·부분 갱신·파생 결과 정합	오류·유실·stale 정책 ___
확장 · 분석/감지	지표 정답 대조·timer 감지	필드 매핑·임계값·지연 ___
Discovery · TS/RAG	비교·평가 결과로 포함 여부 결정	고객 데이터·평가자 ___

측정하지 않은 값은 미측정으로 남기고, 목표는 실행 전에 고객과 합의합니다

DB calls/s와 이벤트/s를 따로 측정합니다

성능 실험은 기능 실습을 통과한 뒤 별도 실행합니다. 아래는 단계적 측정 제안입니다.

01
기능 1회
키·값·순서 정합

→

02
대표 비율
batch·hot key 보존

→

03
점진 부하
관측 → 단계 증가

→

04
지속·복구
lag·retry·정합 대조

기록: unique events/s · DB writes/s · A→C fan-out · p50/p95/p99 · lag · CPU/IOPS · reject/retry

작은 데이터의 explain 실행시간으로 11k calls/s 운영 적합성을 확정하지 않습니다

초기 sizing은 데이터 크기와 키 편향으로 계산합니다

호출량 하나로 Tier·Shard 수를 고정하지 않습니다. 시작 환경의 실측값을 수집합니다.

입력	실측할 것	산출 / 판단
문서 크기	Entity별 avg / p95 BSON · index bytes	상태 / 이력 용량 분리
트래픽	peak · batch · 동시성 · 파생율	CPU · IOPS · lag 관측
키 분포	deviceId cardinality · hot device·resource	샤드 균형·경쟁
쿼리 라우팅	device 기반 조회 vs 모델/지역 집계	targeted / scatter-gather
유일성	hashed index unique 불가 · prefix 제약	샤드키+unique 설계 동시 검증

current-state 개수는 고유 복합키 수. 이벤트 저장량은 unique event/s × 보존 기간입니다

실험 한 건은 환경과 증거를 함께 저장합니다

같은 query shape · 데이터 · 일관성 · 부하로 baseline과 후보를 비교합니다.

항목	기록 템플릿
환경	MongoDB version ___ / Tier ___ / region ___ / data bytes ___
조건	batch ___ / concurrency ___ / read · write mix ___ / w · readPref ___
정합	원천 입력 ___ / unique ___ / 상태 키 ___ / 불일치 ___
성능	ops/s ___ / p95 ___ / lag ___ / docs · keys examined ___
판정	합의 목표 ___ / 통과 · 보류 ___ / 증거 경로 ___

Golden Workload별 Test ID · query · index · 결과 · 성능 log · 결정 이유를 남깁니다

오늘 결정할 항목을 한 장으로 모읍니다

설계안의 선택 이유와 미결 항목을 남겨 후속 PoV의 재작업을 줄입니다.

결정	선택 / 합의 내용	Owner / Due
범위·모델	기기·Entity·기간 / resource 문서 경계 ___	___ / ___
갱신·정합	missing/null/[] · 순서 · A→C 원자성 ___	___ / ___
Golden Workload	A/B W1 · C W2 · R1 · 이력·집계 선택 ___	___ / ___
목표·환경	부하 · p95 · lag · 보존 · 비용 한도 ___	___ / ___
Discovery	TS ___ / RAG ___ / 승격 조건 ___	___ / ___

범위·의미·수치가 미확정이면 Owner와 Due를 지정하고 보류 이유를 기록합니다

첫 실행과 다음 리뷰의 일정을 확정합니다

자료를 다시 요청하기보다, 이미 받은 구조에서 부족한 실행 입력을 구체화합니다.

Owner	Action	Due 합의
Samsung Data / DBA	선정 기기 원본 JSON · DDL · reader baseline · 추출 manifest	___
Samsung App / Platform	필드 · 코드 · 순서 · 배치 · 파생 · 재처리 계약과 topic 준비	___
BI / Business / AI	Golden Query · 감지 의미 · 매뉴얼 · 정답 질문 · 평가자	___
MongoDB SA	모델 · 인덱스 · replay · 측정 runner · 초기 sizing 정리	___
Joint	실행 착수 ___ / 중간 점검 ___ / 결과 리뷰 ___	___

세션 종료 전: 고객 · MongoDB 공동 실행일과 결과 리뷰일을 캘린더에 확정합니다

Entity A–H의 원래 복합키를 보존합니다

stateKey는 canonical JSON 문자열 예시입니다. key 필드에는 원래 키를 함께 저장합니다.

Entity	stateKey 규칙 (deviceId 별도)	추가 확인
A / D / G / H	[Entity, unitId]	Entity로 리소스 의미 구분
B	[B, unitId, component, filterName]	필터별 독립 상태
C	[C, subDevice]	진단 파생 경로
E	[E, unitId, id]	기기 정보 키
F	[F, unitId, target]	OTA 대상 키

unitId의 타입·leading zero·null·경로 의미를 고정하고 변하는 상태값은 키에서 제외합니다

MongoDB 근거와 고객 입력의 출처

공식 근거는 Glean으로 확인했습니다. 각 신규 장표의 발표자 노트에 상세 출처와 진행 메모가 있습니다.

영역	참고 문서 (클릭)
고객 입력	Data_Structure.md §1–5 / 1차 Workshop / Workshop Agenda
문서 모델	MongoDB Data Modeling
갱신·정합	updateOne / Atomicity and Transactions
배치·인덱스	Bulk Write Operations / Unique Indexes
측정	Analyze Query Plans

고객 실측 · 기존 교육용 결과 · 설계 제안 · 내일 실행 결과를 구분해 해석합니다

적재 · 시계열 · 검색의 지원 조건을 확인합니다

Atlas · Connector · Migrator · MongoDB 버전과 실행 환경의 지원 범위를 함께 확인합니다.

영역	공식 문서 (클릭)
이관	Data Migration / Data Verification
Kafka 적재	Sink Write Model Strategies / Document ID Strategies
S3 통합 조회	Atlas Data Federation
Time Series	8.0 Limitations / Automatic Removal (TTL)
Vector Search	Run Vector Search Queries / Prefiltering

링크는 문서 제목을 클릭해 열 수 있으며, 전체 URL은 발표자 노트에 있습니다

Thank you!